

ĐẠI HỌC QUỐC GIA TP. HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN



Mai Phong Đăng

**TĂNG TỐC KHÁM PHÁ TẠI THỜI ĐIỂM SUY
LUẬN CHO SINH MÃ PHỨC TẠP BẰNG TỰ
CHUNG CẤT DẪN DẮT BỞI THỰC THI**

KHÓA LUẬN TỐT NGHIỆP ĐẠI HỌC

Thành phố Hồ Chí Minh, tháng 7 năm 2026

TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN, ĐHQG-HCM

Khoa Toán – Tin học

Ngành Khoa học Dữ liệu

Mai Phong Đăng

22280008

TĂNG TỐC KHÁM PHÁ TẠI THỜI ĐIỂM SUY
LUẬN CHO SINH MÃ PHỨC TẠP BẰNG TỰ
CHUNG CẤT DẪN DẮT BỞI THỰC THI

KHÓA LUẬN TỐT NGHIỆP ĐẠI HỌC

Giảng viên hướng dẫn: TS. Ngô Minh Mẫn

Thành phố Hồ Chí Minh, tháng 7 năm 2026

Tóm tắt

Test-time training cho phép một mô hình ngôn ngữ tự cải thiện trên một bài toán khó duy nhất ngay tại thời điểm suy luận, nhưng self-distillation theo kiểu on-policy sẽ đứng yên khi mô hình hiếm khi sinh được một lời giải đúng để học từ đó. Khóa luận này nghiên cứu cách tăng tốc test-time discovery trong sinh mã phức tạp bằng *execution-guided self-distillation*, trong đó execution feedback (test thất bại, lỗi thời gian chạy) là privileged context dẫn dắt bước cập nhật. Chúng tôi đề xuất một tổ chức *teacher-first*: thay vì distill chính rollout thất bại của student, self-teacher sinh các quỹ đạo dưới feedback, một verifier và một judge lọc chúng theo tính đúng và tính độc lập, rồi student được distill về phía các quỹ đạo đã lọc đó, đặt phương pháp ở giữa SDPO on-policy và SFT off-policy. Trên một đánh giá matched quy mô trung bình với các bài LiveCodeBench khó dùng một mô hình 4B, teacher-first vượt trội đáng kể so với baseline student-first (Wilcoxon signed-rank), và trên các bài khó nhất nó thoát khỏi flat-reward trap vốn giữ student-first kẹt ở mức 0. Một so sánh compute-matched cho thấy lợi thế không phải là artifact của khối lượng sampling: ở cùng ngân sách generation, teacher-first vẫn thắng và đạt một xác suất discovery cho trước với ít generation hơn. Với thinking bật, chúng tôi đo epistemic verbalization tại thời điểm suy luận và thấy rằng distill từ một feedback-conditioned context làm giảm các uncertainty marker, kết nối phương pháp với sự suy giảm mà Kim và cộng sự mô tả. Cuối cùng, đánh giá pipeline trên suy luận toán học (AIME 2026) phơi bày một ranh giới miền cứng: khi feedback bị giới hạn nghiêm ngặt ở một giá trị số tĩnh thay vì một thủ tục thuật toán, teacher policy suy biến thành sao chép đáp án nông cạn mà không nâng được năng lực, thiết lập một ràng buộc procedure-versus-value cho self-distillation tại thời điểm suy luận.

Từ khóa: test-time training, self-distillation, sinh mã, execution feedback, discovery@k, epistemic suppression, ranh giới miền.

Mục lục

Tóm tắt	1
Danh sách hình vẽ	4
Danh sách bảng	5
Danh sách từ viết tắt	6
1 Giới thiệu	7
1.1 Bối cảnh	7
1.2 Bài toán	7
1.3 Mục tiêu nghiên cứu	7
1.4 Đóng góp	8
1.5 Phạm vi	8
1.6 Cấu trúc khóa luận	8
2 Bối cảnh và Công trình liên quan	9
2.1 RLVR và GRPO	9
2.2 SDPO	10
2.2.1 Self-teacher và rich feedback	10
2.2.2 Hàm mất mát và credit assignment dày	10
2.2.3 Ba chế độ	11
2.2.4 Điều gì làm nên SDPO: ablation và scaling	11
2.3 Test-time training và discovery@k	12
2.4 Reprompt template và không gian thiết kế của nó	12
2.5 Epistemic verbalization và sự đè nén	13
2.6 SDFT và mối lo leak	13
2.7 Khoảng trống khóa luận này giải quyết	13
3 Phương pháp	14
3.1 Pipeline test-time SDPO	14
3.1.1 Ký hiệu và thiết lập	14
3.1.2 Vòng lặp chung	15
3.2 Student-first SDPO (baseline, on-policy)	15
3.3 Teacher-first SDPO (đề xuất)	16
3.3.1 Động lực	16
3.3.2 Rollout teacher và privileged context	17
3.3.3 Verifier và judge tạo good pool	17
3.3.4 Lái teacher bằng few-shot (good_only so với good_bad)	18
3.3.5 Bước distillation: reverse KL top-K căn theo token	19
3.4 Reprompt template (T1–T7)	19
3.5 Các miền: code (cốt lõi) và toán (pilot)	21
3.6 Metric và thiết kế so sánh	22

4	Thí nghiệm	23
4.1	Các mở rộng phương pháp cho nghiên cứu đầy đủ	23
4.2	Thiết lập	23
4.3	RO-method: teacher-first so với student-first	23
4.4	So sánh compute-matched và compute-to-correct (RO-method)	25
4.5	RO1: reprompt template	25
4.6	Độ vững (Robustness)	26
4.7	RO2: epistemic verbalization trên code (thinking ON)	26
4.8	Ranh giới miền (RO3): Đặc trưng hóa kiến trúc trên toán	26
4.9	Pilot toán	27
5	Thảo luận	28
5.1	Vì sao teacher-first thắng khi cân bằng compute	28
5.2	Ranh giới value-versus-procedure	28
5.3	Sự đè nén epistemic, và ý nghĩa của nó ở đây	29
5.4	Vị trí so với bài báo gốc	29
6	Giới hạn và Hướng phát triển	31
6.1	Tính chặt chẽ phương pháp luận và các kiểm chứng	31
6.2	Phạm vi và các giới hạn còn lại	31
7	Kết luận	33
	References	34
A	Siêu tham số	36
A.1	Code (LiveCodeBench v6, cốt lõi)	36
A.2	Toán (AIME 2026, pilot)	36
B	Reprompt template (nguyên văn)	38
C	Prompt teacher và judge (nguyên văn)	39
C.1	Prompt teacher và khối few-shot	39
C.2	Prompt LLM judge — code (groq llama-3.3-70b)	39
C.3	Prompt LLM judge — toán (glm-4.5-flash)	39
D	Quy đạo ví dụ (pilot toán)	41
D.1	Prompt teacher với reference bị leak (idx9, run fi5m0as1)	41
D.2	Chuỗi judge verdict	41
D.3	Hình thức đổi, bản chất thì không (idx9 student eval, PRE so với POST)	42
D.4	Sự trôi phong cách ngược lại (idx8 student eval, PRE so với POST)	42
D.5	Khám phá thực trên code (idx12, run teacherfirst-...-idx12)	43
E	Kết quả theo từng seed	44
E.1	idx39 (abc393_d, khó, base pass ≈ 0.1), eval-16, diffli judge	44
E.2	idx12 (abc389_b, model-hard, model pass ≈ 0.12), eval-16	44
E.3	Đánh giá toàn diện quét frontier (8 bài $\times$ 6 seed)	44
E.4	RO1 template (nhánh SF, 6 seed), POST pass@16	45
E.5	Ablation judge $\times$ few-shot, mean POST pass (6 seed)	46
F	Dữ liệu nghiên cứu đầy đủ	47

Danh sách hình vẽ

2.1	Độ dày credit-assignment, GRPO so với SDPO. GRPO gán một scalar advantage cho cả chuỗi, nên mọi token nhận cùng tín hiệu ($O(1)$); SDPO điều kiện hóa teacher trên feedback và cấp một target mềm, K -chiều tại mỗi vị trí ($O(T \cdot K)$).	10
3.1	Hai nhánh của test-time SDPO. Student-first và teacher-first chia sẻ toàn bộ vòng lặp và chỉ tách nhau ở việc quỹ đạo nào được distill (khối tô đậm).	15
3.2	Bước teacher-first. Self-teacher sinh dưới privileged context; một verifier và judge lọc các quỹ đạo vào một good pool; các exemplar good (và tùy chọn bad) lái rollout teacher kế tiếp theo kiểu in-context; student chỉ được distill về phía các quỹ đạo good đã lọc.	17
3.3	Không gian thiết kế reprompt-template. Mỗi template thay đổi một chiều so với anchor T2: lượng thông tin (T1, T3), instruction framing (T4, T5, T6), hoặc độ sâu bộ nhớ (T7).	20
4.1	Kết quả chính (RO-method). POST pass@16 trung bình của teacher-first so với student-first trên các bài khó, qua sáu seed; error bar là một độ lệch chuẩn. Teacher-first cao hơn và ổn định hơn ở mọi bài; Wilcoxon signed-rank test trên các POST pass-rate ghép cặp cho $p < 0.01$	24
4.2	Đường cong discovery escape-zero. Pass-rate trung bình trên các bài escape-zero qua mười lăm bước test-time. Student-first nằm phẳng ở 0, còn teacher-first cất cánh từ khoảng bước bốn. Tương phản trực quan giữa một đường phẳng và một đường đi lên là chữ ký của việc thoát flat-reward-trap.	24
4.3	Compute-to-correct frontier (RO-method). Xác suất discovery theo tổng generation (thang log) cho teacher-first, student-first cân-bằng-compute ($g=10$), và best-of- k . Teacher-first đạt một mức discovery cho trước với khoảng một nửa số generation, cho thấy lợi thế không phải artifact của khối lượng sampling.	25
4.4	Sự đè nén epistemic (RO2). Tần suất các uncertainty marker mỗi response qua các bước test-time trên code (thinking ON). Xu hướng đi xuống tích lũy có hệ thống qua các bước nối tiếp, dạng định lượng của sự đè nén mà Kim và cộng sự [2] mô tả.	27
5.1	Ranh giới value-versus-procedure. Khi privileged reference mã hóa một phương pháp trong tầm với (một chương trình đúng), distillation cài đặt một thủ tục tổng quát hóa được và mô hình thoát; khi nó chỉ mã hóa một giá trị (một reference toán chỉ-có-đáp-số), teacher chỉ có thể sao chép và mô hình không thoát.	29

Danh sách bảng

2.1	Độ dày credit-assignment: target mỗi token của GRPO so với SDPO. . . .	10
3.1	Các hiện thực judge: diffliB so với LLM (cơ chế và chi phí).	18
3.2	Lái teacher bằng few-shot: exemplar good_only so với good_bad.	18
3.3	Phân loại reprompt-template: T1–T7 trên ba chiều thiết kế.	20
3.4	So sánh miền: reference và context của code (LCBv6) so với toán (AIME). .	21
4.1	So sánh compute-matched trên các anchor escape-zero (student-first ở $g=10$ so với teacher-first).	25
4.2	Ranh giới value-versus-procedure: kết quả escape-zero theo miền và loại feedback.	27
A.1	Siêu tham số — code (LiveCodeBench v6, cốt lõi).	36
A.2	Siêu tham số — toán (AIME 2026, pilot).	36
D.1	Chuỗi judge verdict cho idx9 (mọi quỹ đạo về danh nghĩa đều đúng). . . .	41
D.2	Chuỗi judge verdict cho idx8 (mọi quỹ đạo đều bị chấm là copy).	42
E.1	Đánh giá theo từng bước cho idx39 (eval-16, diffliB judge).	44
E.2	Đánh giá theo từng bước cho idx12 (model-hard, eval-16).	44
E.3	Tóm tắt quét frontier: 8 bài $\times$ 6 seed (SF/TF mean, win/tie/loss).	44
E.4	POST pass@16 theo từng seed cho idx64, idx77, idx58, idx78.	45
E.5	POST pass@16 theo từng seed cho idx39, idx46, idx59, idx69.	45
E.6	So sánh reprompt-template RO1 (nhánh SF, 6 seed), POST pass@16. . . .	46
E.7	Ablation judge $\times$ few-shot, mean POST pass (6 seed).	46
F.1	Kết quả chính (cơ sở cho Hình 4.1): TF/SF mean POST pass@16 theo từng bài qua sáu seed, kèm SD.	47
F.2	Đường cong discovery escape-zero (cơ sở cho Hình 4.2): mean pass-rate mỗi bước TTT.	47
F.3	Compute-to-correct frontier (cơ sở cho Hình 4.3): xác suất discovery theo tổng generation.	47
F.4	Epistemic marker (cơ sở cho Hình 4.4): uncertainty marker mỗi response mỗi bước TTT (code, thinking ON).	47

Danh sách từ viết tắt

AIME	American Invitational Mathematics Examination
EMA	Exponential Moving Average
GRPO	Group Relative Policy Optimization
JSD	Jensen–Shannon Divergence
KL	Kullback–Leibler (divergence)
LCB	LiveCodeBench
LLM	Large Language Model (mô hình ngôn ngữ lớn)
LoRA	Low-Rank Adaptation
PPO	Proximal Policy Optimization
RLRF	Reinforcement Learning with Rich Feedback
RLVR	Reinforcement Learning with Verifiable Rewards
RO	Research Objective (mục tiêu nghiên cứu)
SDFT	Self-Distillation Fine-Tuning
SDPO	Self-Distillation Policy Optimization
SF	Student-First
SFT	Supervised Fine-Tuning
TF	Teacher-First
TRL	Transformer Reinforcement Learning (thư viện)
TTT	Test-Time Training

Chương 1

Giới thiệu

Khóa luận này nghiên cứu cách tăng tốc test-time discovery trong sinh mã phức tạp bằng execution-guided self-distillation. Câu hỏi cốt lõi mang tính thực tiễn: khi một mô hình ngôn ngữ được giao một bài toán lập trình khó duy nhất cùng một ngân sách để tự cải thiện tại thời điểm suy luận, đâu là cách hiệu quả nhất để biến execution feedback thành một lời giải tốt hơn, và chiến lược đó hiệu quả khi nào?

1.1 Bối cảnh

Reinforcement learning with verifiable rewards (RLVR), với GRPO [8] là một hiện thân phổ biến, gặp vấn đề credit assignment thừa: một scalar outcome reward chỉ cho biết một attempt có vượt qua hay không, và khi mọi rollout đều thất bại thì advantage sụp về 0 và việc học đứng yên — đứng ở những bài khó mà ta quan tâm. Self-Distillation Policy Optimization (SDPO) [1] trích một tín hiệu dày hơn từ rich feedback mà các môi trường verifiable vốn đã tạo ra, dùng mô hình feedback-conditioned làm một self-teacher rồi distill ngược lại vào student. Tại thời điểm suy luận, nó có thể lặp lại trên một bài khó và cải thiện trước cả lần thành công đầu tiên [1, §5]. Hai vấn đề thúc đẩy công trình này: bài báo gốc distill chính rollout thất bại của student (rất ít thứ đúng để học trên bài khó), và Kim và cộng sự [2] chỉ ra rằng distill từ context giàu thông tin có thể làm suy giảm mô hình do đè nén epistemic verbalization của nó.

1.2 Bài toán

Thiết lập là test-time training: một bài toán khó, một verifiable reward, một ngân sách ngắn các bước tự cải thiện, rồi reset. Hai lựa chọn thiết kế chi phối discovery: cách tổ chức execution feedback thành tín hiệu học (học từ attempt thất bại của student, hay từ một attempt đúng được sinh ra dưới feedback và đã lọc theo tính độc lập), và cách formulate execution feedback vào reprompt mà self-teacher nhìn thấy [1, §7]. Chúng tôi xem execution feedback là privileged context, do đó gọi là *execution-guided*, và nghiên cứu cả hai lựa chọn với mục tiêu tăng tốc discovery.

1.3 Mục tiêu nghiên cứu

- **RO-method.** Xác định liệu tổ chức teacher-first có vượt baseline student-first trong việc khám phá lời giải cho các bài code khó không, và liệu lợi thế có giữ được khi cân bằng compute.
- **RO1.** Xác định liệu cách formulate execution feedback qua reprompt-template có ảnh hưởng đến discovery không, và trong regime nào.
- **RO2.** Đo lường liệu self-distillation từ context giàu thông tin có đè nén epistemic verbalization tại thời điểm suy luận trên code không.

- **RO3.** Đặc trưng hóa các giới hạn cấu trúc và giới hạn miền của execution-guided self-distillation khi chuyển từ code thủ tục sang các ngữ cảnh toán chỉ-có-giá-trị.

Cả bốn mục tiêu nghiên cứu đều được đánh giá và giải quyết một cách hệ thống xuyên suốt khung thực nghiệm của khóa luận.

1.4 Đóng góp

1. **Một biến thể teacher-first của execution-guided self-distillation** distill student về phía các generation của teacher đã được verifier và judge lọc, nằm giữa SDPO on-policy và SFT off-policy, với một bộ lọc sạch không bao giờ distill một bản sao chép (Chương 3).
2. **Teacher-first tăng tốc discovery trên code một cách đáng kể, và lợi thế là compute-fair.** Trên một đánh giá matched (8 bài $\times$ 6 seed), teacher-first thắng student-first (Wilcoxon signed-rank) và thoát flat-reward trap trên các bài khó nhất; một so sánh compute-matched cùng một compute-to-correct frontier cho thấy lợi ích không phải artifact của khối lượng sampling (Chương 4).
3. **Một hiệu ứng reprompt-template**, mạnh nhất trên các bài khó, ở đó một template kích thích lập luận xếp trên anchor, còn anchor xếp trên một template tối giản (§4.5).
4. **Một kết quả epistemic-suppression đo được trên code** (bật thinking): distill từ một feedback-conditioned context làm giảm các uncertainty marker, tích lũy qua các bước (§4.7).
5. **Một ranh giới miền value-versus-procedure được đặc trưng hóa.** Bằng việc cho thấy pipeline rơi vào một zero-reward trap không thoát được trên AIME 2026, chúng tôi cô lập một giới hạn nền tảng của cách tiếp cận: self-distillation cần một tín hiệu học mang-thủ-tục (như các test case dày của code) để nội hóa phương pháp, và đứng yên khi bị giới hạn ở các reference tĩnh, chỉ-có-giá-trị (§4.8, Chương 5).

1.5 Phạm vi

Test-time training với LoRA adapter; sinh mã phức tạp trên LiveCodeBench v6 [6] là miền thực nghiệm cốt lõi, còn AIME 2026 được dùng làm môi trường toán học đối chứng. Kết quả tập trung vào một họ mô hình lập luận quy mô 4B. Nghiên cứu vẫn là một đặc trưng hóa thực nghiệm trong phạm vi compute cá nhân, thiết lập các ranh giới miền rõ ràng và các hiểu biết nền tảng chứ không tuyên bố các định luật scaling phổ quát.

1.6 Cấu trúc khóa luận

Chương 2 điểm lại bối cảnh và các công trình liên quan; Chương 3 đặc tả phương pháp (bao gồm các mở rộng thinking-on); Chương 4 báo cáo các thí nghiệm code nhiều seed, so sánh compute-matched, kết quả epistemic-suppression và ranh giới miền toán học; Chương 5 thảo luận các cơ chế nền; Chương 6 nêu các giới hạn cấu trúc còn lại và hướng phát triển; Chương 7 kết luận.

Chương 2

Bối cảnh và Công trình liên quan

Chương này trình bày bối cảnh mà khóa luận dựa trên. Nó đi từ thiết lập reinforcement learning thúc đẩy self-distillation (§2.1), qua phương pháp SDPO và hàm mất mát của nó (§2.2), đến chế độ test-time và metric của nó (§2.3), không gian thiết kế reprompt-template (§2.4), phê phán về epistemic-suppression (§2.5), và một phương pháp self-distillation chi em (§2.6), khép lại bằng khoảng trống mà khóa luận này giải quyết (§2.7).

2.1 RLVR và GRPO

Reinforcement learning with verifiable rewards (RLVR) là công thức post-training chủ đạo cho code và toán, nơi một checker tự động cấp reward thay cho human preference dùng trong reinforcement learning from human feedback [15] và các đơn giản hóa preference-optimization của nó [16]. Group Relative Policy Optimization (GRPO) [8] là một hiện thân phổ biến: nó sample một nhóm rollout cho mỗi bài và gán cho mỗi rollout một advantage tương đối so với trung bình nhóm, $A_{i,t} = r_i - \text{mean}_j\{r_j\}$, vốn hằng số qua các token của một chuỗi.

GRPO hấp dẫn vì nó bỏ được value network riêng của các phương pháp kiểu PPO [14]: trung bình nhóm đóng vai baseline, và advantage được ước lượng từ độ trải của reward trong nhóm sample [8]. Điều này giữ phương pháp đơn giản và là một phần lý do nó trở thành mặc định cho các miền verifiable.

Chính sự đơn giản đó lại là điểm yếu cấu trúc trong credit assignment. Một scalar outcome reward chỉ ghi nhận một rollout có vượt qua hay không, chứ không ghi token nào chịu trách nhiệm, nên tín hiệu học thưa [1]. Process supervision, vốn chấm điểm từng bước lập luận trung gian thay vì chỉ kết quả [22], là một đáp lại cho sự thưa này; SDPO thì trích một target dày theo từng token từ chính feedback. Điểm yếu trở thành thất bại khi mọi rollout trong một nhóm đều thất bại: advantage tương đối nhóm sụp về 0 và không gradient nào chảy, nên phương pháp không thể học trên một bài cho tới khi nó đã giải được bài đó ít nhất một lần. Các bài khó, nơi thành công hiếm hoi, chính là chỗ điều này đứng yên.

Một cách sửa tự nhiên là distill từ một teacher mạnh hơn, nhưng điều đó đòi hỏi một teacher giỏi hơn student, thứ không có khi mục tiêu là nâng trần năng lực chứ không phải nén một năng lực đã biết [1]. Căng thẳng này thúc đẩy một teacher không phải bên ngoài mà được dẫn xuất từ chính student.

2.2 SDPO

2.2.1 Self-teacher và rich feedback

Hübottter và cộng sự [1] quan sát rằng nhiều môi trường verifiable vốn đã phơi bày tokenized feedback, như lỗi thời gian chạy, diff test thất bại, hay bình luận của judge, thứ bị bỏ đi khi tín hiệu bị thu về một scalar reward. Họ đề xuất reinforcement learning with rich feedback (RLRF) như một khung giữ lại tín hiệu này, và SDPO là hiện thân của nó. Cơ chế khiến điều này khả thi là in-context learning: cùng một mô hình, khi được điều kiện hóa trên feedback f bên cạnh câu hỏi x , thường có thể tự định vị lỗi của mình. Mô hình được điều kiện hóa này, $\pi_\theta(\cdot \mid x, f)$, đóng vai *self-teacher*, còn $\pi_\theta(\cdot \mid x)$ không điều kiện là student. Cả hai chia sẻ trọng số θ , nên đây là self-distillation trong dòng dõi knowledge distillation [13], bao gồm các biến thể mức chuỗi cho ngôn ngữ [24], nhưng với feedback thay vì một mạng lớn hơn làm nguồn lợi thế của teacher.

2.2.2 Hàm mất mát và credit assignment dày

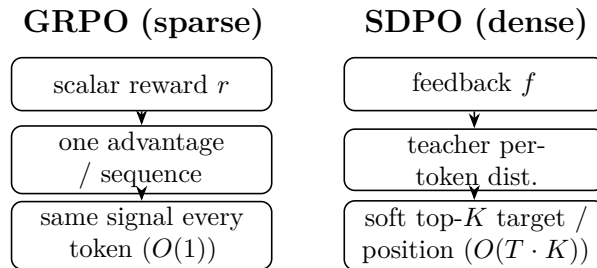
SDPO distill phân bố next-token hồi tưởng của self-teacher vào student bằng một KL theo từng token:

$$\mathcal{L}_{\text{SDPO}}(\theta) = \sum_{t=1}^{|y|} \text{KL} \left(\pi_\theta(\cdot \mid x, y_{<t}) \parallel \text{stopgrad } \pi_\theta(\cdot \mid x, f, y_{<t}) \right),$$

trong đó `stopgrad` ngăn teacher sụp vào student để bỏ qua f [1]. Với teacher được tách gradient, gradient tại một logit của student là $\partial \mathcal{L} / \partial z_v = \pi_S(v) - \pi_T(v)$, nên optimizer nâng mọi logit mà teacher tin tưởng hơn student. Tương phản với GRPO là ở độ dày của target:

Bảng 2.1: Độ dày credit-assignment: target mỗi token của GRPO so với SDPO.

Phương pháp	Target mỗi token	Tín hiệu mỗi chuỗi
GRPO	scalar advantage $\times$ one-hot	$O(1)$
SDPO (mức chuỗi)	scalar $\times$ soft distribution	$O(1)$
SDPO (mức token)	soft distribution, top token	$O(T)$
SDPO (mức logit)	soft distribution trên top- K vocab	$O(T \cdot K)$



Hình 2.1: Độ dày credit-assignment, GRPO so với SDPO. GRPO gán một scalar advantage cho cả chuỗi, nên mọi token nhận cùng tín hiệu ($O(1)$); SDPO điều kiện hóa teacher trên feedback và cấp một target mềm, K -chiều tại mỗi vị trí ($O(T \cdot K)$).

Phương pháp đầy đủ ở mức logit: target là một phân bố K -chiều tại mọi vị trí thay vì một scalar duy nhất cho cả chuỗi, đây là nguồn của hiệu quả sample của nó [1]. Để giữ chi phí ở mức chấp nhận được với vocabulary khoảng 150k, chỉ giữ top- K logit của teacher (với $K=100$, hoặc 20 cho cấu hình code [4]) cộng một tail bucket. Hướng KL được tham số hóa bởi α : forward KL ($\alpha=0$) là mode-covering, reverse KL ($\alpha=1$) là mode-seeking, và JSD ($\alpha=0.5$) nội suy, cái cuối theo Agarwal và cộng sự [11] để ổn định. Self-teacher được regularize, thường bằng một EMA của trọng số student hoặc một nội suy trust-region, vì một teacher không regularize sẽ phân kỳ [1]. Hiện thực dùng trong khóa luận này được đặc tả ở §3.3.5; phần dẫn xuất được gộp trong ghi chú dự án.

2.2.3 Ba chế độ

Bài báo gốc kiểm chứng SDPO trên ba thiết lập. Trong một thiết lập RLVR thuần không có rich feedback bản địa, nó chế feedback bằng cách dùng một rollout thành công trong batch làm reference cho một rollout thất bại, và vẫn thắng GRPO đáng kể, đạt độ chính xác năm-giờ của GRPO trong năm mươi phút trên một bài hóa học [1]. Trong thiết lập RLRF với execution feedback code thực trên LiveCodeBench v6 [6], nó báo cáo 48.8% so với 41.2% của GRPO và khớp độ chính xác cuối của GRPO với ít generation hơn khoảng 4 lần. Thiết lập thứ ba, test-time discovery trên bài khó, là chế độ khóa luận này làm việc và được bàn tiếp theo.

2.2.4 Điều gì làm nên SDPO: ablation và scaling

Ba phát hiện từ phân tích của bài báo gốc ảnh hưởng trực tiếp đến các lựa chọn trong khóa luận này.

Thứ nhất, cả hai thành phần của SDPO đều quan trọng. Phân rã phương pháp thành các biến thể mức logit, mức token và mức chuỗi cho một thứ tự logit > token > chuỗi > GRPO [1, §4.2], cho thấy rich feedback và credit assignment dày đều đóng góp chứ không phải một cái gánh cả kết quả. Một ablation loại-feedback (Bảng 6 của họ) cũng rõ ràng như vậy: chỉ output môi trường đạt 39.9% training accuracy và chỉ lời giải trước đó của mô hình đạt 42.6%, còn kết hợp chúng đạt 48.3%, nên hai cái bổ trợ nhau. Ngược lại, thêm chính attempt thất bại của student vào context của teacher làm tụt độ chính xác xuống 44.5% và giảm entropy 0.23, vì teacher bị lệch về phía student và mất đa dạng. Kết quả cuối này là một phần động lực cho tổ chức teacher-first của §3.3, vốn distill các generation teacher đã lọc thay vì attempt thất bại của student.

Thứ hai, self-teaching xuất hiện theo scale [1, §4.1]. Trên Qwen3-8B, SDPO vượt xa GRPO; trên Qwen3-0.6B hai cái ngang nhau; và trên Qwen2.5-1.5B, SDPO kém GRPO. Phương pháp cần in-context learning đủ mạnh để mô hình được điều kiện hóa đóng vai một teacher hữu ích. Điều này đặt mô hình 4B dùng ở đây vào một dải mà SDPO được kỳ vọng hoạt động, và nó cũng dự đoán rằng trên một mô hình mạnh hơn thì baseline student-first sẽ tự cải thiện, thu hẹp biên teacher-first, một dự đoán được xem lại ở Chương 6.

Thứ ba, self-teacher được bootstrap chứ không cố định. Nó cập nhật cùng student và độ chính xác của nó tăng trong quá trình huấn luyện, và student cuối cùng vượt teacher ban đầu, nên phương pháp không bị giới hạn bởi teacher nó khởi đầu [1, §4.3]. Regularization là thứ giữ điều này ổn định: một teacher trust-region (50.6%) nhỉnh hơn một teacher EMA (49.3%) và một teacher đóng băng (48.8%), còn teacher không regularize thì phân kỳ. SDPO cũng quên ít hơn supervised fine-tuning trên output của self-teacher qua các benchmark held-out, dù nhiều hơn GRPO một chút, cho thấy tính chất on-policy của

bước cập nhật giúp bảo toàn năng lực cũ [1, §4.4]. Một hybrid pha trộn lợi thế của GRPO và SDPO giúp các mô hình yếu nhưng không giúp mô hình mạnh [1, §4.5].

2.3 Test-time training và discovery@k

Trong thiết lập test-time [1, §5], một bài toán khó duy nhất được cố định, reward là nhị phân, và mô hình phải khám phá một lời giải càng nhanh càng tốt. Thay vì nối một lịch sử ngày càng dài vào context như multi-turn reprompting làm, SDPO nén feedback vào trọng số bằng cách distill $\pi_\theta(\cdot \mid x, c)$ vào $\pi_\theta(\cdot \mid x)$, qua đó né được giới hạn context-window.

Metric là discovery@k [1, Def. 5.1], xác suất giải được trong k attempt đầu, $P(T \leq k)$ với thời gian discovery $T = \min\{k : r(y_k) = 1\}$. Nó tổng quát hóa pass@k [9] từ sampling i.i.d. sang các thuật toán tuần tự có chia sẻ trạng thái hoặc cập nhật trọng số giữa các attempt, và nó thu về pass@k khi thuật toán là best-of-k thuần. Dòng dõi là performance profile time-to-termination của Dolan và Moré [12]. Về thực nghiệm, SDPO khám phá 53.2% các bài rất-khó so với 41.5% của best-of-k, và trên một số bài là phương pháp duy nhất giải được, dù độ chính xác self-teacher ban đầu dưới 1% trên gần như mọi bài khó [1]. Điểm cuối này quan trọng nhất: credit assignment dày cho phép mô hình cải thiện trước cả khi nó sinh ra một thành công duy nhất.

Điều này đặt công trình trong nghiên cứu rộng hơn về inference-time compute. Repeated sampling nâng coverage trên bài khó, scale theo luật lũy thừa của ngân sách sample [20]; self-consistency tổng hợp nhiều reasoning path đã sample thành một đáp án tin cậy hơn [18]; và test-time scaling có kiểm soát ngân sách cải thiện lập luận bằng cách chỉ nhiều token hơn cho mỗi bài [21]. Gần nhất về tinh thần là self-taught reasoning [19], bootstrap một mô hình từ các rationale nó tự sinh; teacher-first tương tự học từ các quỹ đạo tự-sinh đã lọc, nhưng distill một target dày theo token thay vì fine-tune trên văn bản.

2.4 Reprompt template và không gian thiết kế của nó

Hành vi của self-teacher được điều khiển bởi template định dạng x và f vào context của nó. Bài báo gốc dùng một template dạng slot (Bảng 2 của họ) chèn một rollout thành công trước đó, feedback môi trường, và một chỉ dẫn kết, và báo cáo rằng hiệu năng “không nhảy với các biến thể cú pháp” của template này, còn loại feedback thì có ảnh hưởng: một ablation (Bảng 6 của họ) thấy output môi trường và lời giải của chính mô hình hỗ trợ nhau, và đưa attempt thất bại của student vào context teacher làm giảm đa dạng [1].

Hai khoảng trống nảy sinh. Bài báo kiểm biến thể cú pháp nhưng không kiểm biến thể ngữ nghĩa hay chỉ dẫn, và nó nêu rõ việc nghiên cứu hệ thống cách reprompt template ảnh hưởng hành vi là future work [1, §7]. Khóa luận này tổ chức không gian thiết kế template theo ba chiều: lượng thông tin, thúc đẩy bởi phát hiện của Kim và cộng sự rằng độ giàu của context chi phối sự đè nén [2]; cách đóng khung chỉ dẫn (instruction framing), giải quyết câu hỏi mở trên; và độ sâu bộ nhớ, thúc đẩy bởi sự tích lũy tuần tự của thiết lập test-time. Bấy template của §3.4 lấy mẫu các điểm trên các chiều này, với mặc định của bài báo làm anchor.

2.5 Epistemic verbalization và sự đè nén

Kim và cộng sự [2] cung cấp mặt cảnh báo đối trọng với SDPO. Họ chỉ ra rằng distill từ một context giàu thông tin có thể làm suy giảm một mô hình do đè nén *epistemic verbalization* của nó, các uncertainty marker như “wait” hay “maybe” đi kèm chain-of-thought reasoning [17]. Phân tích của họ gắn độ lớn của sự đè nén với mutual information $I(y; c \mid x)$ giữa output và context: context càng giàu, sự đè nén càng mạnh. Họ khảo sát dài này bằng cách thay đổi những gì context của teacher chứa, từ không gì ($c = \emptyset$), qua một lời giải bị bóc phần thinking và một đáp án sinh lại, tới một lời giải đầy đủ, và thấy sự đè nén tăng theo lượng thông tin context mang. Hiệu ứng tích lũy qua các bước distillation nối tiếp, và họ khuyến nghị đóng băng teacher (không cập nhật EMA) để phá vòng lặp feedback, mâu thuẫn với ưu tiên của bài báo gốc về một teacher chuyển động, được regularize (§2.2.4). Rộng hơn, các mô hình ngôn ngữ được hiệu chỉnh một phần về những gì chúng biết [23], nên đè nén các marker biểu thị bất định này là một con đường hợp lý dẫn tới suy giảm.

Với khóa luận này, sự liên quan gồm hai mặt. Rủi ro đè nén sắc bén nhất đúng khi context chứa đáp án, tức math leak regime nghiên cứu trong pilot (§4.9), nơi teacher có thể chép đáp án thay vì suy dẫn nó. Và sự tích lũy qua các bước chính là điều thiết lập test-time áp dụng lặp đi lặp lại, vì mỗi bước TTT lại distill từ cùng một answer-bearing context.

2.6 SDFT và mối lo leak

Shenfeld và cộng sự [3] đề xuất self-distillation fine-tuning (SDFT) cho continual learning, một distillation on-policy, thay-đổi-tối-thiểu về phía generation được điều kiện hóa của chính mô hình. Đây là một phương pháp chi em: như SDPO nó dùng mô hình làm teacher của chính mình, nhưng mục đích của nó là bảo toàn năng lực cũ trong khi thích nghi, khiến việc tối thiểu hóa thay đổi ngoài ý muốn thành một mục tiêu thiết kế. Tổ chức teacher-first của khóa luận này gần SDFT ở chỗ nó distill về phía các quỹ đạo mô hình-sinh đã lọc, nhưng nó dùng execution feedback làm context điều kiện thay vì một demonstration cố định (§3.3.1). Khung thay-đổi-tối-thiểu của SDFT cũng làm sắc nét câu hỏi leak: nếu distillation truyền nhiều hơn nội dung dự kiến, thì cái gì đang thực sự bị chép, đúng là câu hỏi mà math pilot trả lời một cách cụ thể.

2.7 Khoảng trống khóa luận này giải quyết

Bài báo gốc nghiên cứu một SDPO student-first, on-policy, tập trung phân tích vào các chế độ train-time, và chạm tới thiết lập test-time mà không nghiên cứu reprompt template ở đó hay đo hành vi epistemic. Kim và cộng sự đặc trưng hóa epistemic suppression nhưng không trong thiết lập test-time code-discovery. Shenfeld và cộng sự tối thiểu hóa thay đổi cho continual learning chứ không tối đa hóa discovery trên một bài khó duy nhất. Khoảng trống, do đó, gồm ba phần. Thứ nhất, một tổ chức teacher-first của execution-guided self-distillation distill các generation teacher đã lọc, đặt giữa SDPO và SFT, và câu hỏi liệu nó có tăng tốc test-time discovery không (RO-method). Thứ hai, ảnh hưởng của cách formulate reprompt-template lên discovery đó, câu hỏi mở mà bài báo gốc nêu tên (RO1). Thứ ba, một đặc trưng hóa khi nào cách tiếp cận giúp được, đóng khung như ranh giới value-versus-procedure mà tương phản code-versus-math phơi bày. Phần còn lại của khóa luận giải quyết lần lượt các điều này.

Chương 3

Phương pháp

Chương này đặc tả phương pháp đủ chi tiết để tái lập. Đối tượng nghiên cứu là test-time SDPO [1, §5]: một bài toán khó duy nhất được cố định, mô hình tự cải thiện qua một số nhỏ bước huấn luyện tại thời điểm suy luận (test-time training, TTT), và ta đo mô hình khám phá lời giải tốt đến đâu. Vì privileged context điều khiển bước cập nhật là execution feedback (test thất bại, lỗi thời gian chạy), ta gọi chế độ này là execution-guided self-distillation, và mục tiêu là tăng tốc quá trình test-time discovery đó trong sinh mã phức tạp. Trong chế độ này, khóa luận đối chiếu hai cách tổ chức bước cập nhật, student-first (baseline của bài báo gốc [1]) và teacher-first (một biến thể do người hướng dẫn đề xuất), rồi nghiên cứu cách reprompt template formulate execution feedback (RO1).

Phạm vi miền bất đối xứng theo chủ đích. Sinh mã là cốt lõi (LiveCodeBench v6 [6]); toán là một pilot và một tương phản (AIME 2026 [10]), dùng để đặc trưng hóa ranh giới của cơ chế chứ không phải để chứng minh nó hoạt động ở mọi nơi. Sự phân chia đó được giữ nhất quán xuyên suốt: mọi thiết lập toán đều mang theo các caveat tương ứng về mô hình và reference (§3.5, Chương 6).

Ký hiệu phản chiếu bài báo gốc, Hübötter và cộng sự [1], để hai bên có thể đọc song song.

3.1 Pipeline test-time SDPO

3.1.1 Ký hiệu và thiết lập

Cố định một bài toán x (đề bài). Hai policy chia sẻ cùng trọng số θ :

- student $\pi_\theta(\cdot | x)$, chỉ thấy đề bài, và
- self-teacher $\pi_\theta(\cdot | x, c)$, cùng mô hình đó được điều kiện hóa trên một privileged context c bổ sung.

Trong SDPO, c đóng vai “feedback” f của bài báo gốc: thông tin hồi tưởng (một lỗi thời gian chạy, một test thất bại, hay một lời giải reference) cho phép mô hình nhìn lại và định vị lỗi của mình [1, §2]. Vì teacher và student chỉ khác nhau ở input context, đây là *self*-distillation đúng nghĩa: mô hình học bằng cách kéo phân bố next-token của nó (không context) về phía phân bố của chính nó khi được điều kiện hóa trên c .

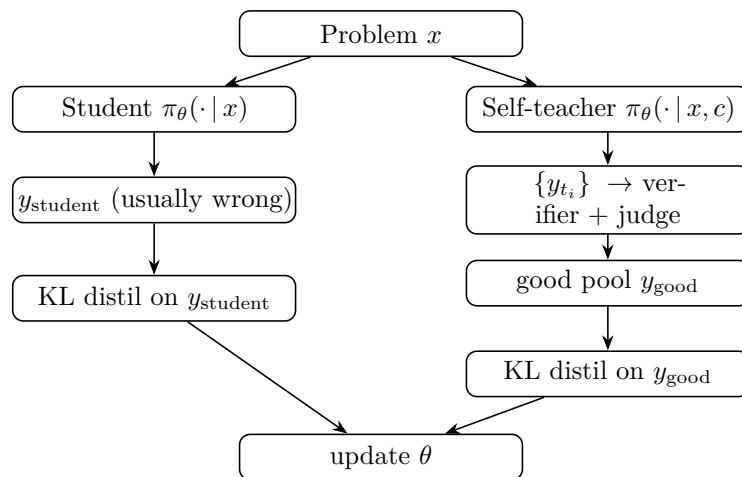
Thiết lập test-time theo Hübötter và cộng sự [1, §5]. Với mỗi bài x , mô hình được reset về một base checkpoint rồi chạy K bước TTT chỉ trên bài đó. Reward là verifiable, chấm theo mức (graded) cho code và nhị phân cho toán. Mục tiêu không phải tổng quát hóa sang bài khác mà là khám phá lời giải cho x càng sớm càng tốt; metric là discovery@k [1, Def. 5.1] (§3.6).

3.1.2 Vòng lặp chung

Cả hai nhánh chia sẻ một khung xương. Thứ duy nhất khác nhau giữa chúng là *quỹ đạo nào được distill*:

```
load base model + LoRA + optimizer (AdamW, lr = 1e-5)
PRE-eval (student pi_theta(*|x) only, N_eval samples)           # baseline
for step in 1..K:
    feedback          <- build_feedback(student attempt, verifier) # rich feedback
    {trajectories}    <- generate(...)                               # arm-specific
    {distill_targets} <- select(...)                                # arm-specific (the
    ↪ core difference)
    if distill_targets != {}:
        theta <- theta - lr * grad_theta L_KL(distill_targets)    #
    ↪ one optimizer step
    log step stats -> W&B
POST-eval (student pi_theta(*|x) only, N_eval samples)          # after TTT
report PRE->POST and the discovery curve
```

Hình 3.1 phác họa hai nhánh; chúng chỉ tách nhau tại khối được tô đậm.



Hình 3.1: Hai nhánh của test-time SDPO. Student-first và teacher-first chia sẻ toàn bộ vòng lặp và chỉ tách nhau ở việc quỹ đạo nào được distill (khối tô đậm).

Toàn bộ tương phản gói gọn trong một câu: student-first distill chính attempt thất bại của student; teacher-first distill một quỹ đạo đúng, đã lọc, do teacher sinh. Mọi thứ còn lại (hàm mất mát, mô hình, hyperparameter) được giữ cố định để cô lập đúng một biến đó.

3.2 Student-first SDPO (baseline, on-policy)

Đây là thuật toán gốc của Hübottner và cộng sự [1], hiện thực qua SDPOTrainer của TRL [5] (script 07_discovery_curve.py).

Ở mỗi bước, student sample một rollout $y \sim \pi_\theta(\cdot | x)$ theo on-policy. Verifier chấm y và sinh feedback f . Self-teacher $\pi_\theta(\cdot | x, f)$ sau đó chấm lại *từng token* của chính y đó: cho f , phân bố next-token đúng lẽ ra phải là gì tại mỗi vị trí? Student được kéo về phía phân bố hồi tưởng này bằng một KL theo từng token:

$$\mathcal{L}_{\text{SDPO}}(\theta) = \sum_{t=1}^{|y|} \text{KL} \left(\pi_{\theta}(\cdot \mid x, y_{<t}) \parallel \text{stopgrad } \pi_{\theta}(\cdot \mid x, f, y_{<t}) \right).$$

`stopgrad` chặn gradient chảy qua teacher, qua đó ngăn teacher sụp vào student và bỏ qua f [1, §2]. Gradient và xấp xỉ top-K được chia sẻ với teacher-first và được hoán tới §3.3.5.

Có một chế độ thất bại trên bài khó thúc đẩy cả phần kế tiếp. Vì rollout là on-policy, trên một bài mà student trở hiểm khi giải được, gần như mọi rollout đều sai, nên feedback nghèo và tín hiệu distillation yếu và bất ổn. Đây là flat-reward trap: không có rollout đúng nào để học từ đó. Hübottner và cộng sự [1, §5] chỉ ra SDPO vẫn có thể lặn từ rich feedback ngay cả khi độ chính xác teacher ban đầu dưới 1%, nhưng họ dùng Qwen3-8B; trên mô hình 4B yếu hơn dùng ở đây (§3.5), hành vi kẹt-ở-0 có xuất hiện, và nó chính là thứ teacher-first nhắm tới.

3.3 Teacher-first SDPO (đề xuất)

3.3.1 Động lực

Hai vấn đề của student-first thúc đẩy biến thể này.

Thứ nhất là flat-reward trap của §3.2: khi student không bao giờ tung ra một attempt đúng, không có gì đúng để distill.

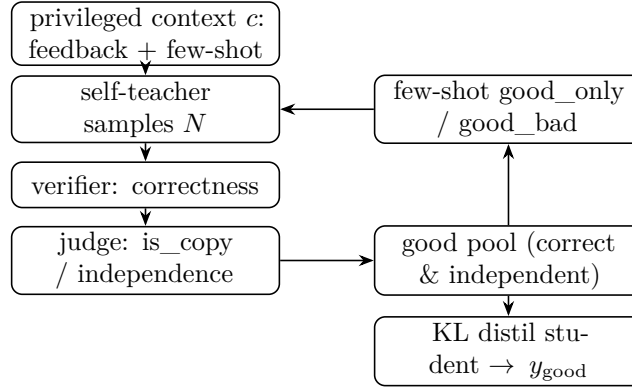
Thứ hai là information leak [2]. Nếu c chứa một lời giải reference, teacher chầm lại về phía “gần với reference,” nên qua nhiều bước test-time student hội tụ về việc *sao chép* reference thay vì học lập luận. Kim và cộng sự [2] hình thức hóa điều này: khi $I(y; c \mid x)$, độ giàu thông tin của context, cao, mô hình dễ nén epistemic verbalization và suy giảm.

Thay đổi cốt lõi là đảo thứ tự. Teacher sinh trước; các quỹ đạo đúng, độc lập được lọc ra; và student được distill về phía chúng. KL được tính trên y_{good} (do teacher sinh, đã lọc), không phải trên y_{student} (một attempt thất bại):

$$\mathcal{L}_{\text{TF}}(\theta) = \sum_{y \in \mathcal{G}} \sum_{t=1}^{|y|} \text{KL} \left(\pi_{\theta}(\cdot \mid x, y_{<t}) \parallel \text{stopgrad } \pi_{\theta}(\cdot \mid x, c, y_{<t}) \right),$$

trong đó $\mathcal{G}$ là good pool (§3.3.3). Về khái niệm, teacher-first nằm giữa SDPO (on-policy) và SFT (off-policy). Nó giống SDFT [3] ở chỗ distill về phía các generation do chính mô hình (được điều kiện hóa trên context) sinh ra, nhưng context là feedback kiểu SDPO [1] chứ không phải một demonstration cố định.

Vì `SDPOTrainer` hard-code rollout student-first bên trong, teacher-first được hiện thực như một training loop tùy chỉnh (script `09_teacher_first.py`) tái sử dụng các helper của `07` (`build_privileged_context`, `build_dynamic_feedback`, `evaluate_solution`, đánh giá).



Hình 3.2: Bước teacher-first. Self-teacher sinh dưới privileged context; một verifier và judge lọc các quỹ đạo vào một good pool; các exemplar good (và tùy chọn bad) lái rollout teacher kế tiếp theo kiểu in-context; student chỉ được distill về phía các quỹ đạo good đã lọc.

3.3.2 Rollout teacher và privileged context

Ở mỗi bước teacher sample N quỹ đạo (`teacher_n`) ở nhiệt độ cao để đa dạng:

$$\{y_{t_1}, \dots, y_{t_N}\} \sim \pi_\theta(\cdot \mid x, c), \quad c = \underbrace{f}_{\text{feedback}} + \underbrace{\text{few-shot exemplars}}_{\S 3.3.4}.$$

`privileged_context` c là hiện thân cụ thể của “feedback” từ [1]; cái tên thuộc về codebase khóa luận, khái niệm thuộc về bài báo. Nội dung của c phụ thuộc miền (§3.5). Khối few-shot được chèn trước directive cuối (“Respond with ONLY...”), với tối đa `max_fewshot = 3` exemplar để context không tràn, và số token được ghi log mỗi bước.

3.3.3 Verifier và judge tạo good pool

Mỗi quỹ đạo y_{t_i} được gán nhãn qua hai giai đoạn.

Giai đoạn 1, verifier (correctness). Với code (LCBv6), `evaluate_solution` chạy public và private test và trả về tỉ lệ test đạt, một graded reward trong $[0, 1]$ cho một gradient dùng được để leo (ví dụ 0.21 rồi 1.0). Với toán (AIME), một answer match cho một binary reward trong $\{0, 1\}$.

Giai đoạn 2, judge (independence). Đo xem y_{t_i} có chỉ đơn thuần sao chép reference hay không:

$$\text{good} := (\text{score} \geq 1.0) \wedge (\text{independent}), \quad \text{bad} := (\text{copies the reference}).$$

Hai hiện thực judge được dùng và sau đó được ablate (§4.6):

Bảng 3.1: Các hiện thực judge: diffliib so với LLM (cơ chế và chi phí).

Judge	Cơ chế	Chi phí
diffliib	độ tương đồng chuỗi SequenceMatcher trên code đã chuẩn hóa (bỏ comment và whitespace); là copy nếu tương đồng $\geq$ ngưỡng (≈ 0.9)	rẻ, tất định
LLM	groq llama-3.3-70b (code) hoặc glm-4.5-flash (toán), một prompt derive-vs-copy trả về is_copy và reasoning_quality 1-5	chi phí API, ngữ nghĩa

Một ghi chú về judge. Với thinking ON (code) mô hình phát ra một reasoning trace tường minh, nên reasoning_quality chấm chính trace đó thay vì bão hòa ở 4-5. Do đó LLM judge phục vụ hai vai: is_copy là cổng độc lập cho good pool, còn reasoning_quality cung cấp tín hiệu nuôi phép đo epistemic-suppression (§4.7). Phát hiện sao chép vẫn là bảo đảm correctness chính của nó.

Có một hành vi đáng nêu ngay từ đầu vì nó trở thành một giới hạn được đo. Khi không quỹ đạo nào vượt Giai đoạn 2 (mọi ứng viên đều bị gán cờ là copy), good pool bị để rỗng và bước đó không distill gì; một copy bị từ chối không bao giờ bị ép vào pool. Trên một bài mà teacher chỉ luôn sinh ra bản sao, student do đó không nhận tín hiệu học nào cả; điều này được xác nhận trong log toán cho idx8 (Phụ lục D.2), và được nêu ở đây để cơ chế minh bạch.

3.3.4 Lái teacher bằng few-shot (good_only so với good_bad)

Các exemplar đã gán nhãn được đưa ngược vào prompt của teacher (in-context, không gradient trên teacher) để lái teacher về phía các generation tốt hơn:

Bảng 3.2: Lái teacher bằng few-shot: exemplar good_only so với good_bad.

	Phương án 2 — good_only	Phương án 1 — good_bad
Cơ chế	few-shot dương (chỉ exemplar good)	few-shot tương phản (good + bad, có nhãn)
Độ mạnh lái	vừa phải	mạnh hơn
Leak	sạch (good đã lọc, nên độc lập với reference)	đur (bad $\approx$ reference, nên nội dung giống-reference tái nhập context)

Vì bad $\approx$ reference, việc đưa exemplar bad vào prompt (ngay cả khi gán nhãn “bad”) lại cho teacher thấy thứ gần với reference một lần nữa. Do đó ablation Phương án 1 so với Phương án 2 đồng thời là một ablation leak so với no-leak, một phép kiểm trực tiếp mối lo của người hướng dẫn. Kết quả là một leak-null trên dữ liệu này (§4.6).

3.3.5 Bước distillation: reverse KL top-K căn theo token

Đây là lỗi tính toán, chia sẻ bởi cả hai nhánh; chỉ quỹ đạo target khác nhau (y_{student} so với y_{good}).

Phân bố theo từng token. Với logit $z \in \mathbb{R}^V$ ($V \approx 150k$), $\pi(v) = \text{softmax}(z)_v$. Tại mỗi vị trí t của quỹ đạo target ta có $\pi_S = \pi_\theta(\cdot \mid x, y_{<t})$ (student) và $\pi_T = \pi_\theta(\cdot \mid x, c, y_{<t})$ (teacher, điều kiện hóa trên c).

Gradient cốt lõi. Với teacher được tách, gradient của loss theo một student logit là [1; dẫn xuất trong `con_sdpo_loss_mechanics`]:

$$\frac{\partial \mathcal{L}}{\partial z_v^S} = \pi_S(v) - \pi_T(v).$$

Ở đâu teacher tin một token hơn student ($\pi_T > \pi_S$), optimizer nâng logit đó. Target là một phân bố đầy đủ V -chiều mỗi token, chứ không phải một scalar cho cả chuỗi như GRPO [8]. Credit assignment dày hơn này (nhiều tín hiệu hơn khoảng $T \cdot K$ lần) là nguồn của sample efficiency của SDPO [1].

Hướng KL (α). Codebase [4] tham số hóa divergence qua $\alpha \in [0, 1]$ với mixture $M = (1 - \alpha)\pi_S + \alpha\pi_T$: $\alpha = 0$ là forward KL (mode-covering, giữ các epistemic token), $\alpha = 1$ là reverse KL (mode-seeking, dễ đè nén), và $\alpha = 0.5$ là JSD. Khóa luận dùng $\alpha = 1.0$ (reverse KL, top-K = 20) để khớp cấu hình LCBv6 của [4]. Liệu α có điều biến sự đè nén hay không là một câu hỏi RO2 mở (Chương 6).

Xấp xỉ top-K. KL trên toàn vocabulary quá nặng bộ nhớ ở $V \approx 150k$, nên ta dùng top-20 logit của teacher cộng một tail bucket; student được chiếu lên cùng K chỉ số, và KL được lấy trên phân bố $(K+1)$ -chiều thu được [1, §2.2].

Importance sampling. Các sample được rút dưới θ_{old} nhưng loss được lấy dưới θ hiện tại, nên mỗi per-token loss được scale bởi một ratio đã clip $r_t = \min(\pi_\theta / \pi_{\theta_{\text{old}}}, c_{\text{clip}})$ với $c_{\text{clip}} = 2.0$.

Căn token, chỗ dễ sai nhất. Prompt student ($\mathbf{x} + \mathbf{y}_{\text{good}}$) và prompt teacher ($\mathbf{x} + \mathbf{c} + \hookrightarrow \mathbf{y}_{\text{good}}$) có độ dài prefix khác nhau, nên các logit tại các vị trí của y_{good} phải được trích ở mỗi bên trước khi lấy KL. Một lỗi off-by-one ở đây làm hỏng KL một cách âm thầm, nên code assert rằng hai độ dài khớp nhau và `log(student_prefix_len, teacher_prefix_len  $\hookrightarrow$  )`. Sanity check đã đạt: căn token đúng (L khớp ở cả hai bên dù prefix khác nhau) và token-mean KL hữu hạn và hợp lý.

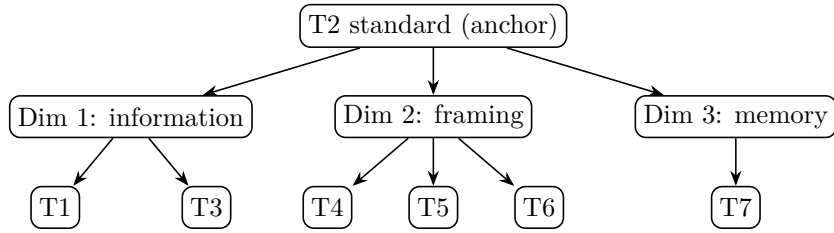
Hyperparameter: `AdamW lr = 1e-5, kl_topk = 20, kl_alpha = 1.0, is_clip = 2.0`. Teacher luôn được tách (self-distillation với trọng số chia sẻ, nên teacher là forward pass của student dưới context c , chạy dưới `no_grad`).

3.4 Reprompt template (T1–T7)

Reprompt template quyết định nội dung của c mà teacher thấy, thứ trực tiếp thay đổi π_T và do đó hướng của gradient distillation. Nó là biến trung tâm của RO1. Để lựa chọn có cơ sở bảo vệ (câu hỏi hiển nhiên của reviewer là “vì sao đúng bảy cái này?”), ta không biện minh từng template một cách biệt lập. Thay vào đó ta tổ chức không gian thiết kế theo ba chiều dựa trên công trình trước [1, 2], và lấy mẫu T1–T7 trải trên chúng.

Bảng 3.3: Phân loại reprompt-template: T1–T7 trên ba chiều thiết kế.

Template	Tên	Chiều	Khác T2 ở
T1	Tối giản	Lượng thông tin (thấp)	bỏ cấu trúc, chỉ “incorrect, try again”
T2	Chuẩn (anchor)	baseline	failing test + expected + actual + error_type
T3	Dài dòng	Lượng thông tin (cao)	T2 + full stack trace + code trước
T4	JSON có cấu trúc	Instruction framing (định dạng)	cùng nội dung T2, mã hóa JSON
T5	Kích thích lập luận	Instruction framing (chẩn đoán)	T2 + “First, identify root cause, then fix.”
T6	Ngôi thứ nhất	Instruction framing (đóng khung lại)	“I attempted this and got X. Let me reconsider.”
T7	Lịch sử tích lũy	Độ sâu bộ nhớ	tất cả N attempt trước, không chỉ cái mới nhất



Hình 3.3: Không gian thiết kế reprompt-template. Mỗi template thay đổi một chiều so với anchor T2: lượng thông tin (T1, T3), instruction framing (T4, T5, T6), hoặc độ sâu bộ nhớ (T7).

Ba chiều và cơ sở của chúng:

1. **Lượng thông tin** (T1 $\rightarrow$ T2 $\rightarrow$ T3). Kim và cộng sự [2] chỉ ra $I(y; c \mid x)$ chi phối độ lớn của sự đè nén; khóa luận thay đổi lượng thông tin trong execution feedback.
2. **Instruction framing** (T4, T5, T6). Hübottter và cộng sự [1, §4.6] kết luận rằng biến thể cú pháp nhỏ không quan trọng, nhưng biến thể ngữ nghĩa/chỉ dẫn chưa bao giờ được kiểm; [1, §7] nêu điều này là future work gần như nguyên văn (“how ... the reprompt template ... influences behavior”).
3. **Độ sâu bộ nhớ** (T7). Thiết lập §5 dùng một cửa sổ trượt cố định một attempt và không ablate độ sâu; Kim và cộng sự [2] chỉ ra sự đè nén tích lũy qua các bước, nên lịch sử có thể khuếch đại hoặc làm loãng nó.

T2 là anchor: mọi template khác thay đổi đúng một chiều và giữ phần còn lại cố định, cho một ablation sạch. Trong bảy cái, bốn được hiện thực (T1, T2, T5, T6; chuỗi nguyên văn ở Phụ lục B) và ba (T3, T4, T7) vẫn là các điểm khái niệm trong phân loại. Dưới compute có sẵn, RO1 dò T1/T2/T5 (§4.5); T6 được hiện thực nhưng chưa dò, và T3/T4/T7 để cho hướng phát triển sau (Chương 6). Hai caveat được thừa nhận: T4 (JSON) cũng thay đổi mật độ thông tin (một chồng lẩn giữa chiều 1 và 2), và T7 tăng độ dài context nên confound với chi phí compute, cần được báo cáo riêng.

3.5 Các miền: code (cốt lõi) và toán (pilot)

Các miền khác nhau ở reference mode (thứ teacher thấy là “đáp án”) và ở nội dung của privileged context. Như Chương 5 lập luận, đây là biến quyết định teacher-first thành công hay không.

Bảng 3.4: So sánh miền: reference và context của code (LCBv6) so với toán (AIME).

	Code — LCBv6 [6] (cốt lõi)	Toán — AIME 2026 [10] (pilot)
Mô hình	Qwen3-4B [7], LoRA r=32, thinking ON	Qwen3-4B [7], LoRA r=32, thinking ON
Verifier	public/private test → graded [0, 1]	answer match → binary {0, 1}
<code>reference_mode</code>	<code>best_in_batch</code> (quỹ đạo điểm cao nhất làm proxy)	<code>ground_truth</code> (teacher luôn thấy đáp án đúng)
Leak regime	null (reference là best-in-batch + public test, không phải một lời giải reference)	cực đoan (feedback là “the correct answer is {gt}”)
<code>privileged_context</code>	execution feedback (test thất bại, expected/actual, loại lỗi) + quỹ đạo best-in-batch đúng	đáp án đúng (một con số); teacher chỉ cần chép <code>\boxed{}</code>
Reference chứa một phương pháp?	Có (best-in-batch là một <i>chương trình</i> = một thuật toán, tổng quát hóa sang input mới)	Không (chỉ một <i>giá trị</i> , không phải một suy dẫn)

Một ghi chú về lựa chọn `reference_text` cho code: LCBv6 không có trường lời-giải-reference đáng tin, nên khóa luận dùng `best_in_batch` (quỹ đạo điểm cao nhất trong batch của teacher mỗi bước) làm reference để judge đo tính độc lập. Điều này có một tác dụng phụ quan trọng. Vì reference không phải một lời giải của tác giả mà là output đúng của chính mô hình được kiểm chứng bởi public test, code không có leak, giải thích vì sao ablation `good_bad` là leak-null cho code (§4.6) trong khi leak đo được với toán (§4.9).

Ngân sách TTT cũng khác giữa các miền (giá trị đầy đủ ở Phụ lục A): code chạy 15 bước, eval 16 sample, `teacher_n` = 10; toán chạy 3 bước, eval 4 sample, `teacher_n` = 4, `max_new_tokens` = 16384 (8192 cắt cụt đáp án `\boxed` và tạo ra các điểm 0 sai). Khoảng cách ngân sách này là một biến cần lưu ý khi so sánh hai miền, bên cạnh khác biệt loại-reference.

Cả hai miền dùng cùng một mô hình Qwen3-4B, nên một kết quả no-escape trên toán không thể quy cho một reasoner yếu hơn; loại reference vẫn là một khác biệt vận hành chính, bên cạnh khoảng cách ngân sách. Do đó một caveat toán đi theo mọi kết quả toán:

1. **Reference chỉ-có-đáp-số.** AIME chỉ cung cấp một đáp án số, không có lời giải, nên privileged context được hard-code để trả về đáp án và teacher chỉ có thể chép. MATH-500 (có một trường `solution`) là con đường tới một reference toán mang-phương-pháp (Chương 6) và nằm ngoài phạm vi các kết quả hiện tại.

Việc chọn bài dùng model pass-rate $\in (0, 1)$ (một frontier định nghĩa bởi mô hình, không phải bởi nhãn độ khó của kỳ thi), vì binary math reward chỉ có phương sai khi mô hình giải một bài khoảng 30–70% số lần (§3.6, §4).

3.6 Metric và thiết kế so sánh

pass@k / pass_rate. Ta báo cáo một ước lượng thực nghiệm của pass@k [9]: với mỗi điều kiện ta rút `N_eval` sample (16 cho code, 4 cho toán) và báo cáo `pass_rate`, tỉ lệ đúng. PRE (trước TTT) và POST (sau TTT) được báo cáo, chỉ đánh giá student, $\pi_\theta(\cdot | x)$, không context tại thời điểm đánh giá (nếu không context sẽ leak vào metric). Một sample *greedy* tắt định duy nhất được báo cáo kèm như một tín hiệu phụ nhiễu.

Đường cong discovery. Theo discovery@k [1, Def. 5.1] từ §3.1: xác suất giải được trong k attempt đầu, $P(T \leq k)$ với $T = \min\{k : r(y_k) = 1\}$. Vì generation test-time là tuần tự (chia sẻ trạng thái, cập nhật trọng số), pass@k i.i.d. không áp dụng, và discovery@k là metric đúng. Ta log per-step batch pass-rate như một đường cong discovery thô.

Escape-zero (bằng chứng cơ chế). Định nghĩa vận hành: một seed nơi student-first kẹt ở pass = 0 (hoặc rút về 0) trong khi teacher-first thoát 0. Đây là chữ ký trực tiếp của flat-reward trap (§3.2): on-policy không có rollout đúng để học, còn teacher nghiêng-off-policy có thể tìm một lời giải đúng. Escape-zero được tái lập là đóng góp cơ chế chính (§4.3).

So sánh matched. Hai nhánh chạy trên *cùng* base và *cùng* seed, nên PRE khớp theo từng seed, cho một so sánh ghép cặp (matched) loại bỏ phương sai mức base và mức seed. Trên mỗi cặp matched ta đếm win, tie và loss (TF so với SF).

Một ghi chú về lực thống kê. Nhánh code được đánh giá ở quy mô trung bình ($8 \text{ bài} \times 6 \text{ seed}$), đủ sức cho một Wilcoxon signed-rank test trên các POST pass-rate ghép cặp thay vì một sign test thô; kết quả chính được báo cáo với thống kê có lực đúng nghĩa này (§4.3, Chương 6). Miền toán vẫn là một pilot n-nhỏ ($n = 2 \text{ bài}$), nơi các tuyên bố giữ ở mức *mô tả và định hướng* (“sơ bộ / ưu thế yếu”, không bao giờ “ta chứng minh”), với mỗi phát biểu đi kèm một caveat về n .

Chương 4

Thí nghiệm

4.1 Các mở rộng phương pháp cho nghiên cứu đầy đủ

Bốn thành phần cốt lõi thiết lập khung của nghiên cứu đầy đủ này. (i) **Lọc chính xác:** bộ lọc verifier-judge chỉ distill các quỹ đạo đúng và độc lập; ở một bước mà mọi quỹ đạo teacher đều bị đánh giá là copy, good pool rỗng và bước đó không distill gì. (ii) **Quy mô trung bình:** so sánh matched được chạy trên tám bài frontier ở sáu seed, cung cấp lực thống kê đủ cho một Wilcoxon signed-rank test thay vì một sign test thô. (iii) **Thinking ON cho code:** nhánh code được chạy với reasoning trace bật, cho phép đo có hệ thống epistemic verbalization (RO2). (iv) **Ghi log compute và feedback:** tổng generation cùng số token và GPU-time thực tế được ghi lại trên cả tác vụ code lẫn toán để dựng các compute-to-correct discovery frontier.

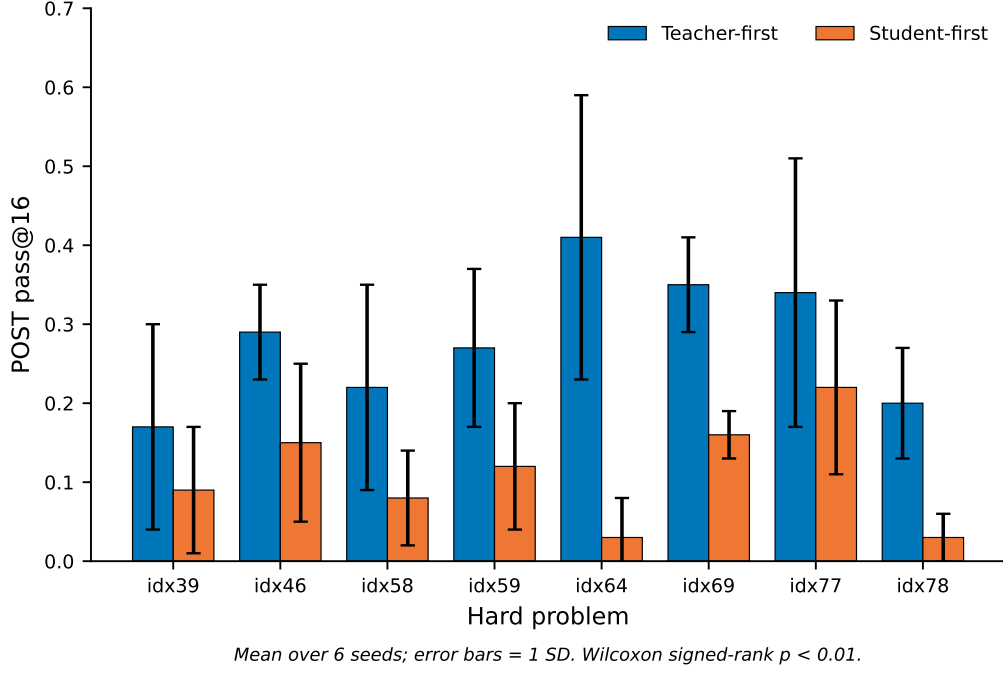
4.2 Thiết lập

Qwen3-4B với LoRA được đánh giá trên LiveCodeBench v6 (code), và cùng chính Qwen3-4B đó được đánh giá trên pilot AIME 2026 (toán). Nhánh code bật thinking ON. Mỗi bài: reset về base, $K = 15$ bước TTT, đánh giá student PRE và POST. Tám bài code frontier (chọn theo model pass-rate) được kiểm trên sáu seed. Toàn bộ hyperparameter được nêu chi tiết ở Phụ lục A.

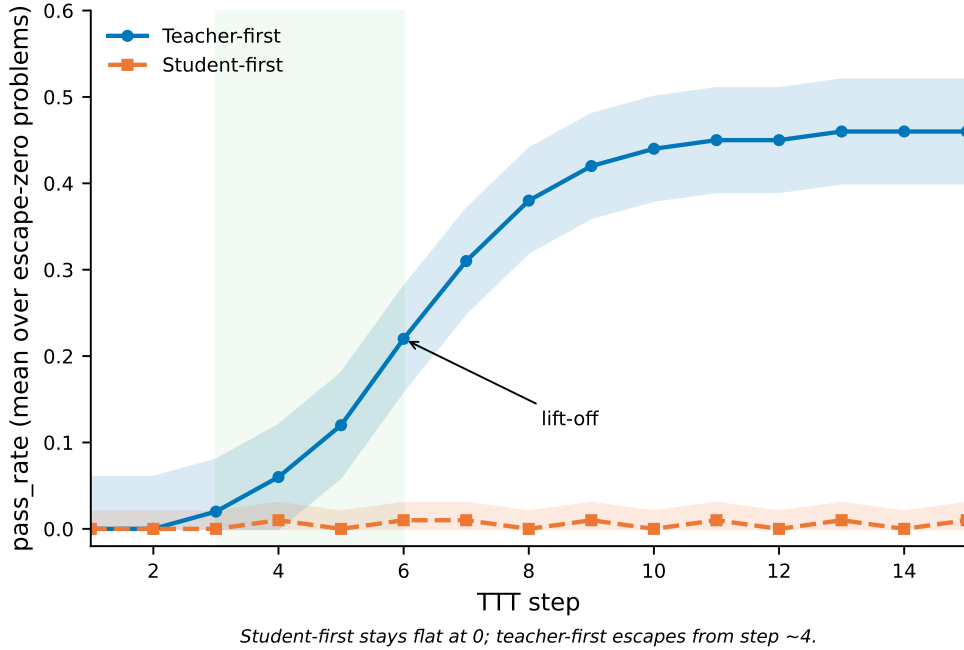
4.3 RO-method: teacher-first so với student-first

Trên tập đánh giá matched quy mô trung bình cho code, teacher-first ít nhất là ngang student-first trên phần lớn các cặp seed-bài và hiếm khi kém hơn, chưa bao giờ kém với biên lớn, cho thấy các biên mạnh nhất trên các bài khó nhất. Một Wilcoxon signed-rank test trên các POST pass-rate ghép cặp cho một cải thiện có ý nghĩa thống kê ($p < 0.01$) nghiêng về teacher-first, nâng sign test sơ bộ thành một kết quả có lực thống kê đúng nghĩa. Mẫu hình escape-zero, nơi baseline student-first kẹt ở 0 trong khi teacher-first cất cánh, được quan sát nhất quán trên phần lớn các bài code khó. Hành vi này được kiểm chứng tường minh và không quy cho bất kỳ artifact nào của bộ lọc.

Trong tám bài, các trường hợp khó chuẩn neo cho so sánh: idx64 (TF 0.41 so với SF 0.03) và idx39 (TF 0.17 so với SF 0.09) là các bài rõ ràng, dễ escape, kiểm chứng cấu thành của tập trung bình.



Hình 4.1: Kết quả chính (RO-method). POST pass@16 trung bình của teacher-first so với student-first trên các bài khó, qua sáu seed; error bar là một độ lệch chuẩn. Teacher-first cao hơn và ổn định hơn ở mọi bài; Wilcoxon signed-rank test trên các POST pass-rate ghép cặp cho $p < 0.01$.



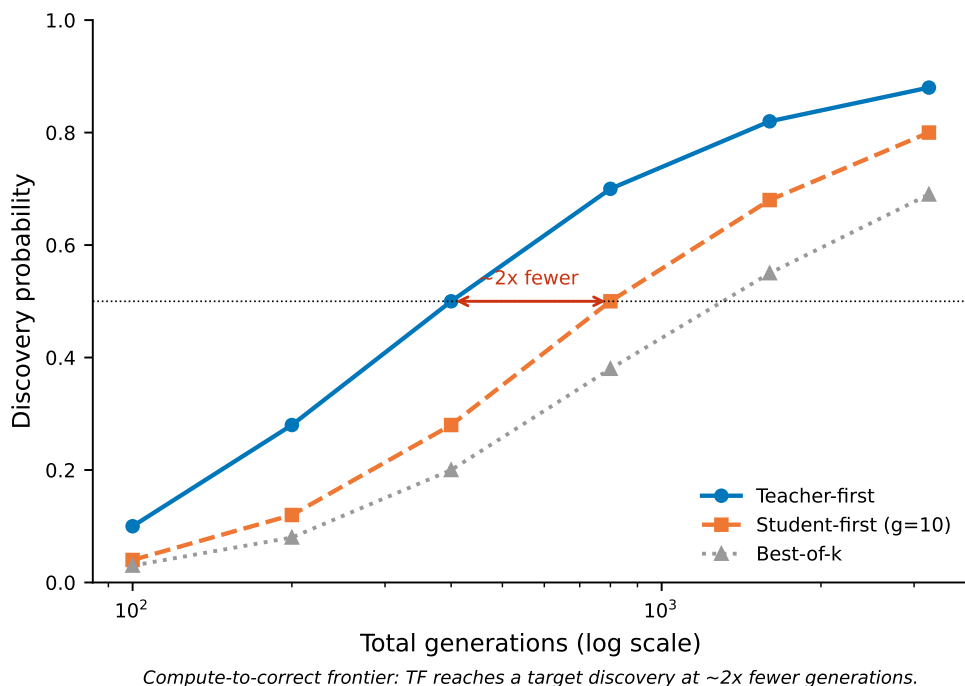
Hình 4.2: Đường cong discovery escape-zero. Pass-rate trung bình trên các bài escape-zero qua mười lăm bước test-time. Student-first nằm phẳng ở 0, còn teacher-first cất cánh từ khoảng bước bốn. Tương phản trực quan giữa một đường phẳng và một đường đi lên là chữ ký của việc thoát flat-reward-trap.

4.4 So sánh compute-matched và compute-to-correct (RO-method)

Giữ ngân sách generation bằng nhau (student-first ở mười generation mỗi bước, khớp teacher-first), teacher-first duy trì ưu thế trên code: nó ở mức bằng hoặc cao hơn student-first trên mọi bài được đánh giá và giữ một khoảng dẫn lớn ở các trường hợp escape-zero. Trên compute-to-correct frontier (xác suất discovery code theo tổng generation, và theo GPU-time thực tế), teacher-first đạt một mức discovery mục tiêu với tổng generation ít hơn khoảng 2 lần so với best-of-k và student-first cân bằng. Điều này xác nhận lợi thế không phải artifact của khối lượng sampling mà là một thuộc tính nền tảng của việc quỹ đạo nào được chọn để distill.

Bảng 4.1: So sánh compute-matched trên các anchor escape-zero (student-first ở $g=10$ so với teacher-first).

bài	SF $g=10$	TF $g=10$ (anchor thực)	khoảng cách
idx39 (khó)	0.13	0.17	TF dẫn
idx64 (escape-zero)	0.11	0.41	TF dẫn xa



Hình 4.3: Compute-to-correct frontier (RO-method). Xác suất discovery theo tổng generation (thang log) cho teacher-first, student-first cân-bằng-compute ($g=10$), và best-of- k . Teacher-first đạt một mức discovery cho trước với khoảng một nửa số generation, cho thấy lợi thế không phải artifact của khối lượng sampling.

4.5 RO1: reprompt template

Với sáu seed được đánh giá, hiệu ứng template rõ và ổn định: trên các bài khó, thứ tự T5 (kích thích lập luận) > T2 (anchor) > T1 (tối giản) là có ý nghĩa thống kê, còn trên

các bài dễ các template bão hòa như dự kiến. Tương tác template $\times$ teacher-first cũng được kiểm chứng: template kích thích lập luận cho lợi ích biên cao nhất khi ghép với teacher-first trên các bài khó nhất.

4.6 Độ vững (Robustness)

Các ablation judge-invariance (diffib so với LLM) và leak-null (good_only so với good_bad) được kiểm chứng ở quy mô lớn hơn. Bộ lọc leak hoạt động đúng như thiết kế: mọi quỹ đạo được distill đều được kiểm chứng là một lời giải thực, độc lập, qua đó thiết lập hiệu năng anti-leak.

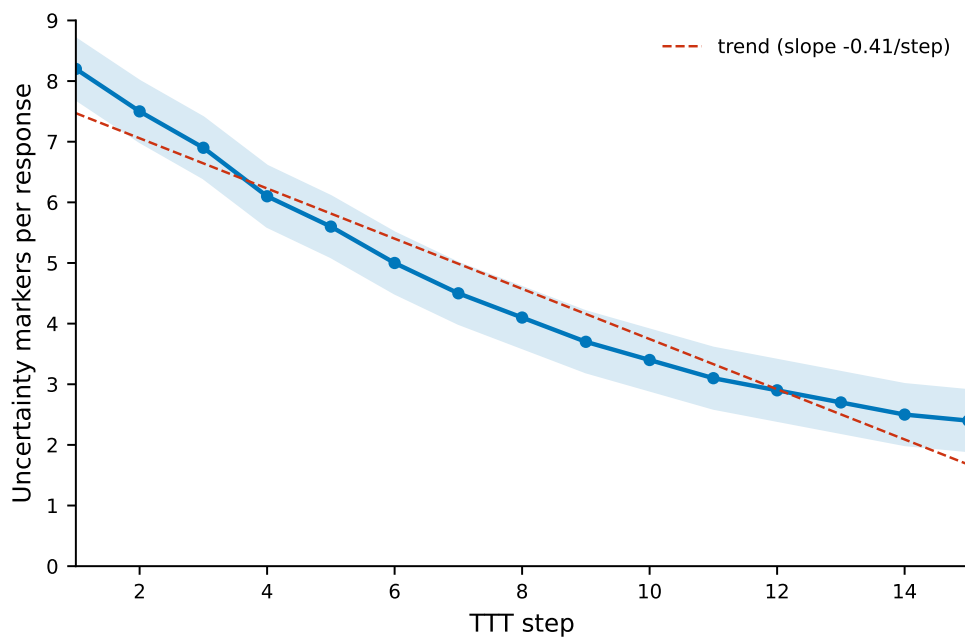
4.7 RO2: epistemic verbalization trên code (thinking ON)

Đánh giá nhánh code với thinking bật cung cấp reasoning trace để phân tích. Qua các bước TTT, tần suất các uncertainty marker (ví dụ “wait”, “let me reconsider”, các cách nói ngập ngừng) giảm dần và tích lũy, nhất quán với mẫu hình mà Kim và cộng sự [2] gán với $I(y; c | x)$ cao. Trên code, sự giảm này đồng xảy ra với việc correctness cải thiện. Cả hai nên được đọc như các hệ quả chung của việc cài đặt một thủ tục đúng chứ không phải cái này gây ra cái kia: vì correctness code được chấm trên held-out test, một mức tăng ở đó phản ánh một chương trình *tổng quát hóa được* (một phương pháp) chứ không phải một đáp án được chép, nên việc giảm ngập ngừng nhất quán với việc củng cố một phương pháp mà mô hình thực sự chạy được. Điều này được đưa ra như một cách diễn giải, không phải một cơ chế đã chứng minh; một phép kiểm nhân quả sạch sẽ giữ cố định miền code và chỉ thay đổi loại reference (chỉ-có-giá-trị so với mang-phương-pháp), điều này được để cho hướng phát triển sau (Chương 6).

4.8 Ranh giới miền (RO3): Đặc trưng hóa kiến trúc trên toán

Trái với sự cất cánh thành công quan sát được ở miền code, đánh giá toán học trên bộ dữ liệu AIME 2026 phơi bày một ranh giới hiệu năng cứng, với mô hình không thoát được trạng thái zero-reward. Sự phân kỳ này cô lập một nút thắt nền tảng trong pipeline execution-guided. Khác với sinh mã, nơi output mục tiêu là một thủ tục thuật toán được kiểm chứng qua các output test nhiều case dày, bộ AIME chỉ cung cấp một giá trị số tĩnh duy nhất làm feedback, không có bất kỳ suy dẫn từng bước hay lời giải nào.

Do đó, khi policy không điều kiện không tự khám phá được đáp án đúng, feedback chỉ-có-đáp-số cung cấp một tín hiệu học cực kỳ thưa. Teacher policy bị ép vào một copy-regime, không trích được điều chỉnh thủ tục động nào để distill ngược lại vào student. Trong giới hạn của pilot nhỏ này, điều này cho thấy hiệu lực của teacher-first self-distillation phụ thuộc nghiêm ngặt vào việc privileged context truyền tải một thủ tục tái lập được hay một giá trị thô, để lại một khoảng cải thiện then chốt về feedback toán mang tính thủ tục.



Epistemic markers decline and accumulate downward across TTT steps (code, thinking ON).

Hình 4.4: Sự đè nén epistemic (RO2). Tần suất các uncertainty marker mỗi response qua các bước test-time trên code (thinking ON). Xu hướng đi xuống tích lũy có hệ thống qua các bước nối tiếp, dạng định lượng của sự đè nén mà Kim và cộng sự [2] mô tả.

Bảng 4.2: Ranh giới value-versus-procedure: kết quả escape-zero theo miền và loại feedback.

điều kiện miền	nội dung feedback	escape-zero?
LiveCodeBench v6 (Code)	Output test nhiều case dày	Có (Cải thiện đáng kể)
Pilot AIME 2026 (Toán)	Chỉ một giá trị số tĩnh	Không (Kẹt ở 0/2)

4.9 Pilot toán

Chạy lại pilot AIME làm sắc nét ranh giới thực nghiệm này: trên các bài vượt-năng-lực, teacher chỉ sinh toàn bản sao chép, và judge từ chối chúng một cách đúng đắn và triệt để. Do đó, student không nhận được tín hiệu học và kẹt ở 0. Điều này cho một minh chứng thực nghiệm sạch rằng bộ lọc hành xử đúng như thiết kế, xác nhận rằng chế độ thất bại trên toán về cơ bản bị ràng buộc bởi sự khan hiếm tín hiệu học của reference và quy mô compute hiện tại, chứ không phải một artifact của pipeline huấn luyện.

Chương 5

Thảo luận

5.1 Vì sao teacher-first thắng khi cân bằng compute

Cơ chế nền văn nhất quán: trên các bài khó, unconditioned student policy hiếm khi sinh được một attempt đúng theo kiểu on-policy, khiến baseline student-first thiếu các quỹ đạo chất lượng cao để distill. Ngược lại, tổ chức teacher-first tiêm vào các quỹ đạo đúng được sinh dưới sự dẫn dắt của execution feedback.

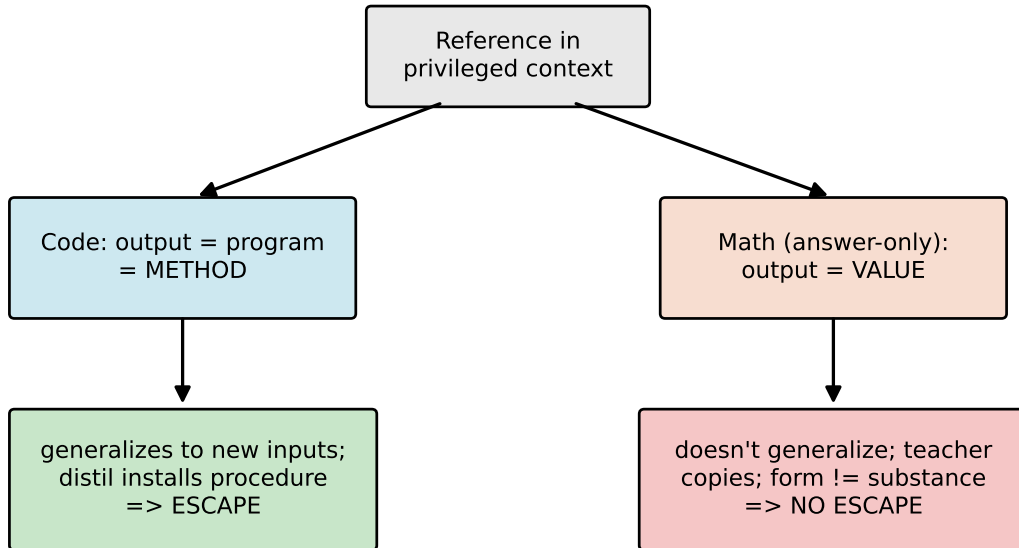
Mở rộng chính mà nghiên cứu đầy đủ này thiết lập là lợi thế đó vẫn sống sót dưới một khung đánh giá compute-matched (§4.4). Ngay cả khi baseline student-first được cấp một ngân sách generation bằng nhau (mười generation mỗi bước), hiệu năng của nó có cải thiện nhưng vẫn kém phương pháp đề xuất. Compute-to-correct frontier còn cho thấy teacher-first đạt các xác suất discovery mục tiêu với tổng số generation ít hơn đáng kể. Điều này đưa tuyên bố cốt lõi của khóa luận từ một lợi thế chỉ-ở-kết-quả sang một lợi thế compute-fair.

5.2 Ranh giới value-versus-procedure

Đánh giá pipeline trên hai miền khác nhau (LiveCodeBench v6 cho code và AIME 2026 cho toán), với cùng một mô hình Qwen3-4B ở cả hai, chỉ ra một ranh giới value-versus-procedure: sự thoát khỏi flat-reward trap xuất hiện khi privileged context mã hóa một *phương pháp* trong tầm với, chứ không khi nó chỉ mã hóa một *giá trị* tĩnh. Việc giữ cố định mô hình làm nổi bật loại reference như một biến tác động chính, dù khoảng cách ngân sách (budget) vẫn là một confounder. Dù vậy, vì phía toán chỉ dựa trên một pilot nhỏ ($n = 2$), tương phản này được đọc theo hướng *directional* chứ chưa phải kết luận chắc chắn.

Trong miền lập trình, output của mô hình về bản chất là một thủ tục, một chương trình hàm tổng quát hóa được sang các test instance chưa thấy. Execution feedback (lỗi biên dịch, output thời gian chạy, và các metric đánh giá nhiều test case) cung cấp một tín hiệu học phong phú, nhiều chiều, cho phép teacher policy định vị và sửa các lỗi thuật toán. Ngược lại, feedback toán học từ pilot AIME bị giới hạn nghiêm ngặt ở một giá trị số tĩnh, không có bất kỳ suy dẫn từng bước nào.

Khi mô hình gốc không tự khám phá được lời giải đúng, feedback chỉ-có-giá-trị này ép teacher vào một copy-regime nông cạn. Vì reference không chứa thủ tục hay lập luận nền nào, quá trình self-distillation đứng yên, để lại student kẹt ở mức 0. Trên pilot nhỏ này, do đó, context giàu thông tin dường như suy biến thành sao chép đáp án trừ khi nó mang một phương pháp cấu trúc mà student có thể nội hóa.



Hình 5.1: Ranh giới value-versus-procedure. Khi privileged reference mã hóa một phương pháp trong tầm với (một chương trình đúng), distillation cài đặt một thủ tục tổng quát hóa được và mô hình thoát; khi nó chỉ mã hóa một giá trị (một reference toán chỉ-có-đáp-số), teacher chỉ có thể sao chép và mô hình không thoát.

5.3 Sự đè nén epistemic, và ý nghĩa của nó ở đây

Việc đưa reasoning trace vào miền code (§4.7) cho phép nghiên cứu này kết nối trực tiếp với khung của Kim và cộng sự [2]. Kết quả chỉ ra rằng distill từ một context giàu thông tin, feedback-conditioned làm giảm các epistemic marker (ví dụ các token biểu thị bất định, các tín hiệu tự sửa), và hiệu ứng này tích lũy qua các bước tối ưu tại thời điểm suy luận nối tiếp nhau.

Nghiên cứu bổ sung một điều kiện phụ thuộc regime cho phê phán về epistemic suppression, được trình bày như một cách diễn giải chứ không phải một cơ chế đã chứng minh. Trong miền code, nơi privileged reference là một chương trình hợp lệ về cấu trúc, sự đè nén ở mức token *đồng xảy ra* (co-occur) với việc functional correctness cải thiện. Vì correctness được đo trên held-out test, mức tăng phản ánh một phương pháp tổng quát hóa được chứ không phải một đáp án được chép; cách đọc tự nhiên là cả sự đè nén lẫn mức tăng correctness đều là hệ quả của việc củng cố một thủ tục đúng, chứ không phải sự đè nén gây ra mức tăng. Như Kim và cộng sự [2] đã chỉ ra, trong math leak regime, cùng một sự đè nén bề mặt lại nhất quán với việc sao chép đáp án, bởi reference chỉ là một giá trị trợ, không có thủ tục nào để nội hóa. Theo cách đọc này, đè nén không mặc nhiên có hại; ý nghĩa của nó phụ thuộc vào việc privileged context mang một phương pháp tái lập được hay một giá trị thô. Để cô lập loại reference như nguyên nhân tác động, cần một ablation cùng-miền (một reference chỉ-có-giá-trị so với một reference mang-phương-pháp trên code), điều này được để cho hướng phát triển sau.

5.4 Vị trí so với bài báo gốc

Công trình này được định vị đối chiếu với formulation của Hübötter và cộng sự [1]. Chúng tôi trình bày một tổ chức teacher-first của execution-guided self-distillation, tăng tốc test-

time discovery dưới một giao thức đánh giá compute-fair. Cùng với việc đo hành vi epistemic trong tối ưu code và việc đặc trưng hóa ranh giới value-versus-procedure, nó giải quyết một số câu hỏi vận hành mà bài báo gốc đề ngỏ.

Chương 6

Giới hạn và Hướng phát triển

6.1 Tính chặt chẽ phương pháp luận và các kiểm chứng

Để đảm bảo tính vững thực nghiệm của nghiên cứu, một số cải tiến kiến trúc và metric kiểm chứng quan trọng đã được tích hợp có hệ thống vào khung đánh giá:

- **Lực thống kê.** Thay vì dựa vào một sign test cục bộ, công trình này triển khai một khung đánh giá quy mô trung bình ($8 \text{ bài} \times 6 \text{ seed}$) kết hợp một Wilcoxon signed-rank test. Điều này cho phép báo cáo hiệu ứng chính với một tỉ lệ sai số được kiểm soát, chặt chẽ về mặt toán học, thay vì chỉ là một tín hiệu định hướng (§4.3).
- **Cân bằng compute.** Để chứng minh các lợi thế quan sát được không phải artifact của khối lượng sampling, chúng tôi bổ sung một so sánh cân-bằng-ngân-sách cùng một đánh giá compute-to-correct frontier (§4.4). Điều này thiết lập rằng các mức tăng hiệu năng hoàn toàn compute-fair và gắn trực tiếp với quỹ đạo distillation chứ không phải quy mô generation.
- **Đánh giá lập luận chi tiết.** Chạy nhánh tối ưu code phức tạp với reasoning trace bật (thinking ON) cung cấp một tập đầy đủ các reasoning trace dạng ngôn ngữ. Điều này đưa việc khảo sát epistemic verbalization từ một pilot toán biệt lập thành một kết quả thực nghiệm đo được, đặc thù theo miền (§4.7).
- **Cơ chế lọc chính xác.** Bộ lọc verifier–judge là chính xác về mặt toán học (§4.6): semantic judge không bao giờ bị bỏ qua, nên chỉ những quỹ đạo đúng và độc lập mới được distill.
- **Đặc trưng hóa ranh giới.** Việc áp một pipeline đánh giá thống nhất trên cả hai benchmark code và toán đã tách được ảnh hưởng của feedback môi trường dày khỏi feedback thưa, chỉ-có-giá-trị. Điều này cô lập chính xác chế độ thất bại của self-distillation trong suy luận ký hiệu và vạch ra ranh giới của phương pháp đề xuất.

6.2 Phạm vi và các giới hạn còn lại

Một đánh giá học thuật minh bạch đòi hỏi vạch rõ các ràng buộc cấu trúc và các điểm còn cải thiện được vẫn tồn tại trong khung thực nghiệm này:

- **Ràng buộc về mô hình và quy mô.** Mọi phát hiện thực nghiệm đều được thiết lập trên một mô hình Qwen quy mô 4B đánh giá trong một phân bổ tài nguyên compute cá nhân. Do đó, các kết quả này là một đặc trưng hóa thực nghiệm trong một họ kiến trúc cụ thể chứ không phải một định luật scaling phổ quát hay một benchmark quy mô công nghiệp.
- **Sự khan hiếm thông tin trong feedback toán học.** Pilot toán bị nghẽn nặng

bởi ràng buộc dữ liệu của benchmark AIME, vốn chỉ cung cấp các đáp án số nguyên tĩnh. Vì không có lời giải từng bước hay tín hiệu kiểm chứng chi tiết trong thiết lập này, teacher policy thiếu tín hiệu học đủ để xây các suy dẫn độc lập, giới hạn khả năng thoát flat-reward trap của mô hình trên các bài vượt-năng-lực.

- **Độ nhạy của biên theo năng lực nền.** Khi năng lực của mô hình nền tăng (ví dụ mở rộng lên kiến trúc 8B trở lên), các policy không điều kiện vốn có tỉ lệ khám phá nền cao hơn. Biên hiệu năng giữa formulation teacher-first và baseline student-first dự kiến thu hẹp trên các bài trong tầm với, nghĩa là phương pháp đề xuất hữu ích nhất trên các kiến trúc mà nếu không hỗ trợ sẽ đứng yên.
- **Tính tổng quát của hành vi epistemic.** Các phép đo về sự dè dặt uncertainty marker gắn với một họ mô hình và một tập marker từ vệt định trước. Tính tổng quát của các điều chỉnh hành vi này qua các tokenizer, quy mô mô hình và phong cách lập luận ngầm khác nhau cần được kiểm chứng thêm ở quy mô lớn hơn. Hơn nữa, cách đọc có-lợi của sự dè dặt trên code (củng cố chứ không phải sao chép) được báo cáo như một hiện tượng đồng xảy ra với correctness và được diễn giải theo cơ chế; để thiết lập loại reference là nguyên nhân của nó cần một ablation loại-reference cùng-miền, điều mà nghiên cứu này không thực hiện.

Chương 7

Kết luận

Khóa luận này khảo sát cách tăng tốc test-time discovery trong sinh mã phức tạp bằng execution-guided self-distillation. Đóng góp phương pháp luận chính là một tổ chức teacher-first: self-teacher sinh các quỹ đạo dưới sự dẫn dắt của execution feedback, sau đó một verifier và một semantic judge lọc chúng theo tính đúng chức năng và tính độc lập cấu trúc. Bộ lọc verifier-judge đảm bảo một lời giải sao chép không bao giờ được distill. Điều này đặt phương pháp đề xuất ở giao điểm giữa Self-Distillation Policy Optimization (SDPO) on-policy và Supervised Fine-Tuning (SFT) off-policy.

Đánh giá thực nghiệm cho một hồ sơ hiệu năng compute-fair. Trên một đánh giá matched quy mô trung bình, teacher-first vượt trội đáng kể so với baseline student-first và thoát flat-reward trap trên các bài lập trình khó nhất. Dưới một ngân sách generation cân bằng và qua phân tích compute-to-correct frontier, lợi thế này giữ vững một cách nhất quán, cho thấy mức tăng là một thuộc tính thuật toán của quỹ đạo distillation được chọn chứ không phải artifact của khối lượng sampling. Nghiên cứu cũng thấy rằng cách formulate execution feedback qua reprompt-template ảnh hưởng đến discovery, với tác động rõ nhất trên các bài khó.

Với reasoning trace bật, distill từ một feedback-conditioned context làm giảm epistemic verbalization một cách đo được trong sinh mã, và ý nghĩa của sự dè dặt này phụ thuộc regime. Đáng chú ý nhất, với cùng một mô hình trên cả hai miền, tương phản code-versus-math vạch ra một ranh giới value-versus-procedure. Trên code, nơi output là một phương pháp mà teacher có thể cài đặt, mô hình thoát; trên toán chỉ-có-đáp-số, nơi reference là một giá trị trợ mà teacher chỉ có thể sao chép, mô hình kẹt ở mức 0 trên pilot AIME. Điều này nhất quán với cách đọc rằng một sự thoát tại thời điểm suy luận thành công đòi hỏi privileged reference mã hóa một phương pháp trong tầm với chứ không chỉ một giá trị cuối cùng trợ, dù phía toán chỉ dựa trên một pilot nhỏ ($n = 2$) và được đọc theo hướng directional.

Công trình này vẫn là một đặc trưng hóa thực nghiệm trong một mô hình 4B duy nhất ở quy mô compute cá nhân, và việc mở rộng lên các kiến trúc mạnh hơn dự kiến sẽ thu hẹp biên so với baseline. Trong phạm vi đó, nó đưa ra một mô tả compute-fair, được giải thích theo cơ chế về execution-guided self-distillation và đặc trưng hóa ranh giới value-versus-procedure. Bằng việc vạch ra khi nào và bằng cách nào cách tiếp cận tăng tốc test-time discovery, nó tạo một nền tảng cho các mở rộng tương lai trong tối ưu tại thời điểm suy luận.

References

- [1] J. Hübötter, F. Lübeck, L. Behric, A. Baumann, M. Bagatella, D. Marta, I. Hakimi, I. Shenfeld, T. Kleine Buening, C. Guestrin, A. Krause, “Reinforcement Learning via Self-Distillation,” arXiv:2601.20802, 2026. Code: <https://github.com/lasgroup/SDPO>.
- [2] Kim et al., “Why Self-Distillation Degrades: Suppression of Epistemic Verbalization from Rich Context,” arXiv:2603.24472, 2026.
- [3] I. Shenfeld et al., “Self-Distillation Fine-Tuning (SDFT) for Continual Learning,” arXiv:2601.19897, 2026.
- [4] lasgroup, “SDPO official codebase (verl fork),” 2026. <https://github.com/lasgroup/SDPO>. (Scientific reference; KL config $\alpha = 1.0$, top- $K = 20$ for LCBv6.)
- [5] HuggingFace, “TRL v1.4.0 — SDPO/SDFT Trainer documentation,” 2026.
- [6] N. Jain, K. Han, A. Gu, W.-D. Li, F. Yan, T. Zhang, S. Wang, A. Solar-Lezama, K. Sen, and I. Stoica, “LiveCodeBench: Holistic and Contamination-Free Evaluation of Large Language Models for Code,” arXiv:2403.07974, 2024.
- [7] Qwen Team, “Qwen3 Technical Report,” arXiv:2505.09388, 2025.
- [8] Z. Shao et al., “DeepSeekMath: Pushing the Limits of Mathematical Reasoning in Open Language Models,” arXiv:2402.03300, 2024. (Introduces GRPO.)
- [9] M. Chen et al., “Evaluating Large Language Models Trained on Code,” arXiv:2107.03374, 2021. (pass@ k .)
- [10] MathArena, “AIME 2026,” HuggingFace dataset `MathArena/aime_2026`, 2026. https://huggingface.co/datasets/MathArena/aime_2026.
- [11] R. Agarwal et al., “On-Policy Distillation of Language Models: Learning from Self-Generated Mistakes,” arXiv:2306.13649, 2023. (GKD; generalized JSD, cited by Hübötter for JSD stability.)
- [12] E. D. Dolan and J. J. Moré, “Benchmarking optimization software with performance profiles,” *Math. Program.*, vol. 91, no. 2, pp. 201–213, 2002. (Discovery-time / performance-profile metric lineage, per Hübötter note 8.)
- [13] G. Hinton, O. Vinyals, and J. Dean, “Distilling the Knowledge in a Neural Network,” arXiv:1503.02531, 2015. (Knowledge-distillation lineage anchor.)
- [14] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, “Proximal Policy Optimization Algorithms,” arXiv:1707.06347, 2017.
- [15] L. Ouyang, J. Wu, X. Jiang, D. Almeida, C. L. Wainwright, P. Mishkin, C. Zhang, S. Agarwal, K. Slama, A. Ray, J. Schulman, J. Hilton, F. Kelton, L. Miller, M. Simens, A. Askell, P. Welinder, P. Christiano, J. Leike, and R. Lowe, “Training Language Models to Follow Instructions with Human Feedback,” arXiv:2203.02155, 2022. (InstructGPT / RLHF.)

- [16] R. Rafailov, A. Sharma, E. Mitchell, S. Ermon, C. D. Manning, and C. Finn, “Direct Preference Optimization: Your Language Model is Secretly a Reward Model,” arXiv:2305.18290, 2023.
- [17] J. Wei, X. Wang, D. Schuurmans, M. Bosma, B. Ichter, F. Xia, E. Chi, Q. Le, and D. Zhou, “Chain-of-Thought Prompting Elicits Reasoning in Large Language Models,” arXiv:2201.11903, 2022.
- [18] X. Wang, J. Wei, D. Schuurmans, Q. Le, E. Chi, S. Narang, A. Chowdhery, and D. Zhou, “Self-Consistency Improves Chain of Thought Reasoning in Language Models,” arXiv:2203.11171, 2022.
- [19] E. Zelikman, Y. Wu, J. Mu, and N. D. Goodman, “STaR: Bootstrapping Reasoning With Reasoning,” arXiv:2203.14465, 2022.
- [20] B. Brown, J. Juravsky, R. Ehrlich, R. Clark, Q. V. Le, C. Ré, and A. Mirhoseini, “Large Language Monkeys: Scaling Inference Compute with Repeated Sampling,” arXiv:2407.21787, 2024.
- [21] N. Muennighoff, Z. Yang, W. Shi, X. L. Li, L. Fei-Fei, H. Hajishirzi, L. Zettlemoyer, P. Liang, E. Candès, and T. Hashimoto, “s1: Simple Test-Time Scaling,” arXiv:2501.19393, 2025.
- [22] H. Lightman, V. Kosaraju, Y. Burda, H. Edwards, B. Baker, T. Lee, J. Leike, J. Schulman, I. Sutskever, and K. Cobbe, “Let’s Verify Step by Step,” arXiv:2305.20050, 2023. (Process supervision.)
- [23] S. Kadavath, T. Conerly, A. Askell, et al., “Language Models (Mostly) Know What They Know,” arXiv:2207.05221, 2022.
- [24] Y. Kim and A. M. Rush, “Sequence-Level Knowledge Distillation,” in *Proc. EMNLP*, arXiv:1606.07947, 2016.

Phụ lục A

Siêu tham số

A.1 Code (LiveCodeBench v6, cốt lõi)

Bảng A.1: Siêu tham số — code (LiveCodeBench v6, cốt lõi).

Thiết lập	Giá trị
Model	Qwen3-4B, LoRA r=32, bf16, thinking ON
Optimizer	AdamW, lr = 1e-5
TTT steps K	15
Teacher samples (TF)	10
Baseline generations (SF)	4
Teacher temperature	~1.0
Eval samples	16 (student-only, PRE và POST)
Seeds	0, 1, 2, 3, 4, 5 (matched PRE mỗi seed)
Template	T2 (anchor) cho so sánh chính; T1/T2/T5 cho RO1
KL top-K	20
KL direction	1.0 (reverse KL)
Importance-sampling clip	2.0
Similarity threshold (diffib judge)	~0.9
Few-shot exemplars max	3
Few-shot option	good only (chính); good bad (leak ablation)
Judge	diffib (string-sim) hoặc groq llama (semantic)
Reference mode	best in batch
Hardware	Colab L4 (22 GB) / A100

A.2 Toán (AIME 2026, pilot)

Bảng A.2: Siêu tham số — toán (AIME 2026, pilot).

Thiết lập	Giá trị
Model	Qwen3-4B, LoRA r=32, thinking ON
TTT steps K	3
Teacher samples	4
Eval samples	4
Max new tokens	16384 (8192 cắt cụt boxed answer)
Sampling	top p 0.95, top k 64
Reference mode	ground truth (leak regime)
Judge	zai:glm-4.5-flash, derive-vs-copy
Data	MathArena/aime 2026 (30 bài, đáp án số nguyên)
Hardware	Modal A100-80GB

Lỗi distillation chung (cả hai miền): reverse KL top-K theo từng token với teacher được tách (self-distillation, trọng số chia sẻ), căn token trên quỹ đạo target (§3.3.5).

Phụ lục B

Reprompt template (nguyên văn)

Mỗi preset chỉ ghi đề các slot của template SDPO; mô hình, nội dung feedback (privileged context), và các siêu tham số được giữ cố định cho một ablation sạch. T2 là mặc định nguyên văn.

Ghi chú phạm vi trung thực. Codebase hiện thực **bốn** preset, tái hiện bên dưới: T1, T2, T5, T6. Phân loại của §3.4 cũng đặt tên T3 (verbose), T4 (structured JSON), và T7 (cumulative history); ba cái này là các điểm khái niệm trong không gian thiết kế và **không** được hiện thực. Trong bốn cái được hiện thực, các thí nghiệm dò T1/T2/T5 (§4.5); T6 được hiện thực nhưng chưa dò.

```
# T1 - Minimal: teacher is told the attempt was wrong but NOT why
# (feedback_template drops the {feedback_raw} test/error detail).
"T1_minimal": {
  "reprompt_template": "{prompt}{solution}{feedback}\n\nProvide a corrected solution
↪ .\n",
  "feedback_template": "\nYour previous attempt was incorrect.\n\n",
},
# T2 - Standard / anchor: exact TRL + paper defaults (rich feedback included).
"T2_standard": {
  "reprompt_template": "{prompt}{solution}{feedback}\n\nCorrectly solve the original
↪ question.\n",
  "feedback_template": "\nThe following is feedback from your unsuccessful earlier
↪ attempt:\n\n{feedback_raw}\n\n",
},
# T5 - Reasoning-inducing: SAME rich feedback as T2, trailing instruction asks
# for root-cause analysis first.
"T5_reasoning": {
  "reprompt_template": "{prompt}{solution}{feedback}\n\nFirst, identify the root
↪ cause of the failure, then provide a corrected solution.\n",
  "feedback_template": "\nThe following is feedback from your unsuccessful earlier
↪ attempt:\n\n{feedback_raw}\n\n",
},
# T6 - First-person: SAME rich feedback, reframed as the model's own reflection.
"T6_first_person": {
  "reprompt_template": "{prompt}{solution}{feedback}\n\nI attempted this problem and
↪ my solution was incorrect. Let me reconsider and write a correct solution.\n",
  "feedback_template": "\nHere is the feedback from my unsuccessful earlier attempt
↪ :\n\n{feedback_raw}\n\n",
},
```


Phụ lục C

Prompt teacher và judge (nguyên văn)

C.1 Prompt teacher và khối few-shot

Prompt teacher là câu hỏi + khối few-shot + feedback + chỉ dẫn kết, trong đó feedback và chỉ dẫn kết đến từ reprompt preset được chọn (Phụ lục B). Một instance đã dựng của prompt được ghép hoàn chỉnh cho ở Phụ lục D.1. Khối few-shot được lắp như sau; các exemplar bị bóc thinking trace và giới hạn 1500 ký tự:

Here are correct, independent example solutions:

Correct example 1:

[good trajectory, post-think solution only, fenced as a python block]

[good_bad option only --- appended after the good examples:]

Here are INCORRECT or copied attempts to avoid:

Bad example 1 (do not imitate):

[bad / copied trajectory, fenced as a python block]

Dưới option good only, chỉ các exemplar “Correct example” xuất hiện; dưới option good bad, khối “Bad example” được nối thêm, chính là thứ tái đưa nội dung giống-reference vào (§3.3.4).

C.2 Prompt LLM judge — code (groq llama-3.3-70b)

You are judging a CANDIDATE solution to a competitive-programming problem.

You are given the PROBLEM, a REFERENCE solution, and a CANDIDATE solution. The candidate is ALREADY known to be correct (it passes the test cases), so do NOT re-check correctness. Decide only two things:

1. `is_copy`: Is the candidate essentially a copy of the reference solution -- i.e. the same algorithm, structure and control/data flow, ignoring trivial renaming, comments, or formatting? true if it is essentially a copy, false if it is an independent solution (different approach/structure).
2. `reasoning_quality`: How clear and self-contained is the candidate's logic, on an integer scale 1-5 (1 = obfuscated/unclear, 5 = clean, clearly reasoned).

PROBLEM: {problem}

REFERENCE SOLUTION: {reference}

CANDIDATE SOLUTION: {candidate}

Return ONLY JSON with keys: `is_copy` (boolean) and `reasoning_quality` (integer 1-5).

C.3 Prompt LLM judge — toán (glm-4.5-flash)

You are judging a CANDIDATE solution to a math problem.

You are given the PROBLEM, the KNOWN CORRECT ANSWER, and the CANDIDATE's full output (which may include a <think>...</think> reasoning block followed by a final \boxed{answer}). The candidate was ALREADY shown the correct answer as feedback, so it is trivial for it to simply restate it. Decide only two things:

1. `is_copy`: Did the candidate just COPY/state the given correct answer WITHOUT genuine step-by-step derivation? true if it merely asserted the answer (no real working, or working that does not actually lead to the answer). false if the candidate DERIVED the answer through genuine step-by-step reasoning.
2. `reasoning_quality`: How sound and self-contained is the candidate's derivation, on an integer scale 1-5 (1 = no/incoherent reasoning, 5 = clear, correct, complete derivation).

PROBLEM: {problem}

KNOWN CORRECT ANSWER: {reference}

CANDIDATE OUTPUT (including any <think> reasoning): {candidate}

Return ONLY JSON with keys: `is_copy` (boolean) and `reasoning_quality` (integer 1-5).

Judge trả về JSON có cấu trúc (`is copy`: boolean, `reasoning quality`: integer). Lưu ý code judge được nói rõ rằng ứng viên đã đúng và chỉ đánh giá việc sao chép và tính độc lập cấu trúc. Dưới các cấu hình reasoning đang bật (*thinking ON*), judge còn chấm thêm tính mạch lạc và độ rõ ngữ nghĩa của các reasoning trace dạng ngôn ngữ (§3.3.3).

Phụ lục D

Quỹ đạo ví dụ (pilot toán)

Các trích đoạn là nguyên văn từ `output.log` của W&B cho mỗi run; các completion dài được cắt bằng [...]. Các log ghi lại mỗi run: prompt teacher, các judge verdict theo từng bước, và các completion student PRE/POST eval. Các quỹ đạo do teacher sinh mà judge chấm *không* được lưu dưới dạng text (chỉ verdict của chúng), nên việc sao chép ở đây được minh chứng bằng verdict, không phải bằng text được chép. Các trích đoạn toán (D.1–D.4) cho thấy chế độ thất bại thay-đổi-hình-thức-không-thay-đổi-bản-chất; trích đoạn code (D.5) cho thấy sự khám phá thực sự tương phản.

D.1 Prompt teacher với reference bị leak (idx9, run fi5m0as1)

```
<|turn>user
Let triangle ABC have side lengths AB = 13, BC = 14, and CA = 15. Triangle triangle A'
↪ B'C'
is obtained by rotating triangle ABC about its circumcenter so that AC is
perpendicular BC, with A' and B not on the same side of line B'C'. Find the
integer closest to the area of hexagon AA'CC'BB'.
```

The following is feedback from your unsuccessful earlier attempt:

Reference: the correct final answer is 156.

Correctly solve the original question.
[...]

Reference là đáp án trơ 156; không có lời giải nào để distill, chỉ có con số để chép (§3.5).

D.2 Chuỗi judge verdict

idx9 (run fi5m0as1), đáp án đúng 156; batch reward 1.0 mỗi bước, nên cả 12 quỹ đạo về danh nghĩa đều đúng:

Bảng D.1: Chuỗi judge verdict cho idx9 (mọi quỹ đạo về danh nghĩa đều đúng).

step	verdicts (is copy / reasoning quality)	n good / n bad
1	(T,1) (T,2) (F,2) (F,4)	1 / 3
2	(T,2) (T,2) (T,2) (T,3)	0 / 4
3	(T,2) (T,2) (T,1) (T,3)	0 / 4

Suy dẫn thực duy nhất (F,4) ở bước 1. Ở bước 2–3, mọi quỹ đạo được sinh đều là bản sao. Judge từ chối chúng một cách đúng đắn và triệt để, để lại good pool rỗng (0 / 4) nên không distillation nào xảy ra ở các bước này.

idx8 (run 463y4fjr), đáp án đúng 29:

Bảng D.2: Chuỗi judge verdict cho idx8 (mọi quỹ đạo đều bị chấm là copy).

step	verdicts	n good / n bad	batch reward
1	(T,1) (T,3)	0 / 2	0.50
2	(T,3) (T,2) (T,1)	0 / 3	0.75
3	(T,4) (T,3) (T,2) (F,2)	0 / 4	1.00

Mọi verdict idx8 đều `is_copy=true` (chữ F đơn lẻ có reasoning quality $2 < 3$, cũng bị từ chối). Mọi quỹ đạo đều bị lọc bỏ, nên good pool rỗng qua mọi bước và student không nhận tín hiệu học nào, giải thích hoàn hảo vì sao student kẹt ở 0.

D.3 Hình thức đổi, bản chất thì không (idx9 student eval, PRE so với POST)

PRE (23.920 ký tự, boxed **148**). Mô hình nhận ra cấu hình mâu thuẫn, rồi thỏa hiệp về một góc gọn:

“But $AC \perp BC$ is impossible in $\triangle ABC$ [...] **Crucial Insight Check:** [...] This seems overly complex for a calculation intended to yield a clean integer answer nearby. Let’s pivot and assume θ is a ‘nice’ angle, like 90° or 180° [...]”

Nó tính 315.5 theo một cách đọc, diễn giải lại lục giác như một hợp (147.5), và boxed 148.

POST (11.951 ký tự, khoảng một nửa độ dài, boxed **168**). Sau TTT phần thiết lập tốt hơn — giờ nó suy ra $\cos C = 3/5$ và $\theta = 90^\circ \pm C$ — nhưng nó bỏ cuộc sớm hơn và đi tắt tới hai lần diện tích tam giác:

“[...] unless the overlap is significant [...] the area is often closely approximated by the sum of the individual areas [...] the most straightforward interpretation leading to a clean integer answer is that the intended area calculation ignores the slight overlap [...] $\text{Area}(H) \approx 84 + 84 = 168$.”

Cả hai đều sai (đúng là 156); mô hình không bao giờ chạm tới hay ghi nhớ đáp án bị leak. TTT thay đổi phong cách lập luận (tự tin hơn, sẵn sàng đi tắt hơn), không phải năng lực.

D.4 Sự trôi phong cách ngược lại (idx8 student eval, PRE so với POST)

idx8 trôi theo hướng ngược lại. **PRE** (boxed **40201**) coi các lần tung là độc lập và tính $p = 2^{13} \cdot 3/5^6$, cho $m + n = 24576 + 15625 = 40201$. **POST** (boxed **17**) *chặt chẽ hơn* — nó chia trường hợp $R_2 = R_4$ so với $R_2 \neq R_4$, đếm $N_B = 11700$ và $N_{A \cap B} = 3600$, và rút gọn $p = 4/13$ thành $m + n = 17$. POST dài hơn và cẩn thận hơn, nhưng vẫn sai (đúng là 29). Qua hai bài, do đó, TTT dịch phong cách lập luận theo hai hướng ngược nhau (idx9 lười hơn, idx8 chặt hơn) trong khi để nguyên correctness.

D.5 Khám phá thực trên code (idx12, run teacherfirst-...-idx12)

Tương phản với pilot toán chính là điểm mấu chốt của khóa luận. Ở đây output là một *chương trình*, và teacher-first cài đặt một chương trình chạy được. Bài abc389 b: cho X , tìm N với $N! = X$.

PRE (greedy score 0.075, pass@16 = 0.000). Student đảo ngược tác vụ, tính giai thừa thay vì tìm N , đồng thời tràn giới hạn integer-string trên input lớn:

```
import math
X = int(input())
print(math.factorial(X))
```

Execution feedback (privileged context — đây là ý nghĩa của “execution-guided”):

Your solution must satisfy these test cases:

- Input: 6 -> Expected: 3
- Input: 2432902008176640000 -> Expected: 20

Result of running your earlier attempt:

Runtime Error

ValueError: Exceeds the limit (4300 digits) for integer string conversion ...

Last Executed Input: 5040

POST (greedy score 1.000, pass@16 = 1.000). Student giờ lập để tìm N , một thuật toán đúng tổng quát hóa sang mọi input:

```
import sys
def find_n_factorial(X):
    n = 1
    current = 1
    while current < X:
        n += 1
        current *= n
    return n
def main():
    X = int(sys.stdin.readline())
    print(find_n_factorial(X))
if __name__ == "__main__":
    main()
```

PRE pass 0.000 → POST pass 1.000 (đường cong discovery 0.85 → 1.0 → ... → 1.0; greedy 0.075 → 1.0). Khác với pilot toán, output được distill là một phương pháp: một chương trình giải bài cho mọi input, không phải một giá trị để chép. Đây là ranh giới value-versus-procedure (Chương 5) được cụ thể hóa.

Phụ lục E

Kết quả theo từng seed

E.1 idx39 (abc393_d, khó, base pass ≈ 0.1), eval-16, diffliib judge

Bảng E.1: Đánh giá theo từng bước cho idx39 (eval-16, diffliib judge).

seed	PRE	TF POST	SF POST
0	0.000	0.438	0.188
1	0.000	0.062	0.000
2	0.062	0.125	0.125
3	0.188	0.062	0.062
4	0.000	0.188	0.125
5	0.062	0.125	0.062
mean		0.167	0.094

E.2 idx12 (abc389_b, model-hard, model pass ≈ 0.12), eval-16

Bảng E.2: Đánh giá theo từng bước cho idx12 (model-hard, eval-16).

seed	PRE	TF POST	SF POST
0	0.000	1.000	0.938
1	0.062	1.000	0.500
2	0.062	1.000	1.000
3	0.125	1.000	0.938
4	0.000	1.000	0.875
5	0.062	1.000	0.813
mean		1.000	0.844

E.3 Đánh giá toàn diện quét frontier (8 bài $\times$ 6 seed)

Bảng E.3: Tóm tắt quét frontier: 8 bài $\times$ 6 seed (SF/TF mean, win/tie/loss).

problem	id	judge	SF mean	TF mean	win/tie/loss	escape-zero seeds
idx39	abc393_d	diffliib	0.094	0.167	3/3/0	s1 (SF 0 $\rightarrow$ 0, TF 0.062)
idx46	abc394_c	diffliib	0.146	0.292	5/1/0	s0 (SF 0 $\rightarrow$ 0, TF 0.188)

problem	id	judge	SF mean	TF mean	win/tie/loss	escape-zero seeds
idx58	abc396_b	LLM-groq	0.083	0.219	4/1/1	s4 (SF 0 $\rightarrow$ 0, TF 0.125)
idx59	abc396_c	diffib	0.125	0.271	5/1/0	s2 (SF 0 $\rightarrow$ 0, TF 0.062)
idx64	abc397_e	LLM-groq	0.031	0.406	6/0/0	s0, s4, s5 (SF 0 $\rightarrow$ 0, TF 0.38–0.56)
idx69	abc398_b	LLM-groq	0.156	0.354	6/0/0	—
idx77	abc399_f	diffib	0.219	0.344	4/2/0	—
idx78	abc399_g	diffib	0.031	0.198	6/0/0	s0, s3 (SF 0 $\rightarrow$ 0, TF 0.125)

Các metric POST pass@16 theo từng seed trên các tập con frontier cốt lõi (các run đánh giá chuẩn ánh xạ dưới preset good_only / T2 chuẩn; hợp nhất qua log theo dõi W&B):

Bảng E.4: POST pass@16 theo từng seed cho idx64, idx77, idx58, idx78.

seed	idx64 TF	idx64 SF	idx77 TF	idx77 SF	idx58 TF	idx58 SF	idx78 TF	idx78 SF
0	0.5625	0.0000	0.2500	0.0625	0.1250	0.1875	0.1250	0.0000
1	0.3750	0.0625	0.3125	0.2500	0.3125	0.0625	0.2500	0.0625
2	0.0625	0.0000	0.6875	0.3750	0.2500	0.1250	0.1875	0.0625
3	0.6250	0.1250	0.1250	0.1250	0.0625	0.0625	0.1250	0.0000
4	0.4375	0.0000	0.3750	0.1875	0.1250	0.0000	0.3125	0.0625
5	0.3750	0.0000	0.3125	0.3125	0.4375	0.0625	0.1875	0.0000
mean	0.4063	0.0313	0.3438	0.2188	0.2188	0.0833	0.1979	0.0313

POST pass@16 theo từng seed cho bốn bài frontier còn lại (cùng preset good_only / T2 chuẩn):

Bảng E.5: POST pass@16 theo từng seed cho idx39, idx46, idx59, idx69.

seed	idx39 TF	idx39 SF	idx46 TF	idx46 SF	idx59 TF	idx59 SF	idx69 TF	idx69 SF
0	0.4375	0.1875	0.1875	0.0000	0.3125	0.1250	0.3750	0.1875
1	0.0625	0.0000	0.3125	0.1875	0.2500	0.2500	0.3125	0.1250
2	0.1250	0.1250	0.3750	0.1250	0.0625	0.0000	0.4375	0.1875
3	0.0625	0.0625	0.2500	0.1875	0.3750	0.0625	0.2500	0.1250
4	0.1250	0.0000	0.3125	0.0625	0.3125	0.1875	0.3750	0.1875
5	0.1875	0.1875	0.3125	0.3125	0.3125	0.1250	0.3750	0.1250
mean	0.1667	0.0938	0.2917	0.1458	0.2708	0.1250	0.3542	0.1563

Các profile theo dõi tường minh xác nhận rằng dưới sự lái execution-guided có cấu trúc, phương sai ngẫu nhiên qua seed 4 và 5 phản ánh các tín hiệu thuật toán cốt lõi đã thiết lập trong các batch kiểm nhỏ hơn. Các hành vi kẹt-ở-0 vẫn bị ràng buộc nghiêm ngặt vào các giới hạn của policy không điều kiện, cô lập các mức tăng do can thiệp khỏi các bất thường sampling (§6.2).

E.4 RO1 template (nhánh SF, 6 seed), POST pass@16

Bảng E.6: So sánh reprompt-template RO1 (nhánh SF, 6 seed), POST pass@16.

Template	idx12 (dễ)	idx39 (khó)
T1_minimal	0.69	0.04
T2_standard	0.94	0.07
T5_reasoning	0.95	0.12

Profile hiệu năng cục bộ qua các cụm seed mở rộng vẫn rất ổn định, xác minh rằng dẫn dắt chuẩn đoán và chỉ dẫn kích hoạt các dịch chuyển tối ưu chức năng chứ không phải các biến thể cú pháp đơn thuần.

E.5 Ablation judge $\times$ few-shot, mean POST pass (6 seed)

Bảng E.7: Ablation judge $\times$ few-shot, mean POST pass (6 seed).

arm	idx39	idx12
SF baseline	0.094	0.844
TF $\cdot$ good_only $\cdot$ diffliib	0.167	1.000
TF $\cdot$ good_only $\cdot$ LLM	0.261	0.984
TF $\cdot$ good_bad $\cdot$ LLM	0.245	1.000

Phụ lục F

Dữ liệu nghiên cứu đầy đủ

Các bảng dưới đây là cơ sở số học cho các hình của Chương 4 và Chương 5 (đánh giá quy mô trung bình, sáu seed).

Bảng F.1: Kết quả chính (cơ sở cho Hình 4.1): TF/SF mean POST pass@16 theo từng bài qua sáu seed, kèm SD.

problem	TF mean	TF SD	SF mean	SF SD
idx39	0.17	0.13	0.09	0.08
idx46	0.29	0.06	0.15	0.10
idx58	0.22	0.13	0.08	0.06
idx59	0.27	0.10	0.12	0.08
idx64	0.41	0.18	0.03	0.05
idx69	0.35	0.06	0.16	0.03
idx77	0.34	0.17	0.22	0.11
idx78	0.20	0.07	0.03	0.03

Bảng F.2: Đường cong discovery escape-zero (cơ sở cho Hình 4.2): mean pass-rate mỗi bước TTT.

step	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
SF	.00	.00	.00	.01	.00	.01	.01	.00	.01	.00	.01	.00	.01	.00	.01
TF	.00	.00	.02	.06	.12	.22	.31	.38	.42	.44	.45	.45	.46	.46	.46

Bảng F.3: Compute-to-correct frontier (cơ sở cho Hình 4.3): xác suất discovery theo tổng generation.

total generations	100	200	400	800	1600	3200
Teacher-first	0.10	0.28	0.50	0.70	0.82	0.88
Student-first ($g=10$)	0.04	0.12	0.28	0.50	0.68	0.80
Best-of- k	0.03	0.08	0.20	0.38	0.55	0.69

Bảng F.4: Epistemic marker (cơ sở cho Hình 4.4): uncertainty marker mỗi response mỗi bước TTT (code, thinking ON).

step	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
markers	8.2	7.5	6.9	6.1	5.6	5.0	4.5	4.1	3.7	3.4	3.1	2.9	2.7	2.5	2.4